

Universidade Federal de Viçosa
Campus Rio Paranaíba

Pedro Augusto Londe Ribeiro - 9340

Análise de Algoritmos de Ordenação

Rio Paranaíba - MG

2025

Universidade Federal de Viçosa
Campus **Rio Paranaíba**

Pedro Augusto Londe Ribeiro - 9340

Análise de Algoritmos de Ordenação

Trabalho apresentado para obtenção de créditos na disciplina SIN213 - Projeto de Algoritmo da Universidade Federal de Viçosa - Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba - MG
2025

1 RESUMO

A análise dos algoritmos de ordenação é essencial no campo da tecnologia da informação, pois permite compreender o comportamento e a eficiência de diferentes métodos aplicados a problemas de organização de dados. O objetivo deste trabalho é comparar o desempenho dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort*, *Shell Sort*, *Merge Sort*, *Quick Sort* e *Heap Sort*, identificando em quais contextos cada um se mostra mais adequado e eficiente. Essa análise contribui para a escolha correta de técnicas de ordenação, visando maior otimização e desempenho dos sistemas computacionais.

Os algoritmos foram implementados na linguagem C++ e testados sob diferentes condições de entrada: crescente, decrescente e aleatória. A pesquisa teve caráter empírico, utilizando o tempo de execução como principal variável de análise, e relacionando os resultados obtidos com as complexidades teóricas dos algoritmos nos melhores, piores e casos médios.

Os resultados indicaram que os algoritmos baseados em divisão e conquista, como o *Merge Sort*, o *Quick Sort* e o *Heap Sort*, apresentaram o melhor desempenho geral, especialmente em grandes volumes de dados. Já os algoritmos de troca, como o *Insertion Sort* e o *Shell Sort*, mostraram-se mais eficientes em conjuntos menores ou parcialmente ordenados, destacando a importância de compreender o contexto de aplicação para selecionar o método mais apropriado.

Sumário

1	RESUMO	1
2	INTRODUÇÃO	5
3	ALGORITMOS	6
3.1	Insertion Sort	6
3.2	Selection Sort	8
3.3	Bubble Sort	11
3.4	Shell Sort	13
3.5	Merge Sort	16
3.6	Quick Sort	18
3.7	Heap Sort	20
3.7.1	Heap	20
3.7.2	Filas de Prioridade	22
4	ANÁLISE DE COMPLEXIDADE	24
4.1	Insertion Sort	24
4.1.1	Forma Geral	25
4.1.2	Melhor Caso	25
4.1.3	Pior Caso	26
4.1.4	Medio Caso	27
4.1.5	Conclusão da Análise do Insertion Sort	28
4.2	Selection Sort	28
4.2.1	Forma Geral	29
4.2.2	Melhor Caso	29
4.2.3	Pior Caso	30
4.2.4	Médio Caso	31
4.2.5	Conclusão da Análise do Selection Sort	32
4.3	Bubble Sort	32
4.3.1	Forma Geral	32

4.3.2	Melhor Caso	33
4.3.3	Médio e Pior Caso	34
4.3.4	Conclusão da Análise do Bubble Sort	35
4.4	Shell Sort	35
4.4.1	Complexidade Shell Sort	36
4.5	Merge Sort	37
4.5.1	Cálculo Complexidade	38
4.5.2	Merge	38
4.5.3	Conclusão da Análise do Merge Sort	40
4.6	Quick Sort	40
4.6.1	Fórmula Geral	42
4.6.2	Melhor Caso	42
4.6.3	Médio Caso	44
4.6.4	Pior Caso	44
4.6.5	Conclusão da Análise do Quick Sort	45
4.7	Heap Sort	46
4.7.1	Conclusão da Análise do Heap Sort	49
5	TABELA E GRÁFICO	49
5.1	Insertion Sort	49
5.2	Selection Sort	50
5.3	Bubble Sort	51
5.4	Shell Sort	52
5.5	Comparação dos Algoritmos Baseado em Trocas	53
5.5.1	Crescente	53
5.5.2	Decrescente	54
5.5.3	Aleatória	55
5.6	Merge Sort	56
5.7	Quick Sort	57
5.7.1	Quick Sort Versão 1	58

5.7.2	Quick Sort Versão 2	59
5.7.3	Quick Sort Versão 3	60
5.7.4	Análise das Três Versões em Conjunto	61
5.8	Comparação dos Algoritmos Baseados em Divisão e Conquista	63
5.8.1	Crescente	64
5.8.2	Decrescente	65
5.8.3	Aleatória	66
5.9	Heap Sort	67
6	CONCLUSÃO	68
7	REFERÊNCIAS BIBLIOGRÁFICAS	70

2 INTRODUÇÃO

Os dados, em suas diferentes formas, estão cada vez mais presentes no cotidiano, tornando-se fundamentais para a organização e a tomada de decisões. Quando desordenados, podem dificultar o acesso à informação, gerar confusões e acarretar desperdício de tempo. Um exemplo simples ocorre em uma lista de chamada: localizar uma matrícula em ordem alfabética é rápido e direto, mas em uma lista desordenada a busca se tornaria confusa e demorada.

Este relatório tem como objetivo analisar e comparar diferentes algoritmos de ordenação. Como afirma que “algoritmos de ordenação têm como objetivo reorganizar um conjunto de registros de modo que fiquem dispostos em uma determinada ordem em função de um campo chave associado a cada registro” (ZIVIANI, 2011), o estudo busca evidenciar em quais situações determinados algoritmos apresentam melhor desempenho em relação a outros.

O trabalho foi desenvolvido no âmbito da disciplina SIN213 – Projeto de Algoritmos da Universidade Federal de Viçosa, utilizando diferentes tipos de entrada para avaliar o desempenho dos métodos estudados. A análise foi conduzida a partir do tempo de execução dos algoritmos, com os resultados apresentados em tabelas e gráficos, de forma a identificar os cenários em que cada abordagem se mostra mais eficiente.

Para melhor organização, o relatório foi estruturado em sete seções: (1) resumo, apresentando uma visão geral do projeto; (2) introdução, contextualizando o tema e seus objetivos; (3) descrição dos algoritmos, acompanhada de exemplos de execução; (4) análise de complexidade, discutindo os diferentes casos e ordens de crescimento; (5) resultados, com a apresentação de tabelas e gráficos que fundamentam a análise; (6) conclusão, sintetizando os achados do estudo; e (7) referências, listando as obras consultadas para embasar o trabalho.

3 ALGORITMOS

Os algoritmos de ordenação representam ferramentas essenciais para a organização de informações, pois permitem estruturar conjuntos de dados de modo a facilitar sua manipulação e análise em diversos contextos. Além de tornar o acesso mais ágil, sua utilização contribui para a otimização de processos computacionais e para a eficiência de sistemas em larga escala (CORMEN et al., 2012).

Existem diversos algoritmos de ordenação, cada um com vantagens e limitações próprias. A eficiência de sua aplicação depende não apenas do tamanho da entrada, mas também de seu estado inicial, podendo variar entre os cenários de melhor, médio e pior caso.

A partir desta seção, serão apresentadas as descrições dos algoritmos analisados, destacando suas principais características, como funcionamento, complexidade e os contextos em que seu uso é mais vantajoso.

3.1 Insertion Sort

O Insertion Sort é um dos algoritmos de ordenação mais simples e intuitivos, sendo frequentemente utilizado como ponto de partida para o estudo de técnicas de ordenação. Sua lógica é semelhante à maneira como organizamos cartas de baralho: a cada passo, seleciona-se um novo elemento e insere-se na posição correta dentro da porção já ordenada do arranjo. Apesar de não ser o algoritmo mais eficiente para grandes conjuntos de dados, ele possui fácil implementação e bom desempenho em listas pequenas ou quase ordenadas, o que justifica sua ampla utilização em contextos didáticos e em aplicações específicas.

De forma mais técnica o Insertion Sort constrói, passo a passo, um prefixo já ordenado do array. A cada iteração, ele seleciona o elemento atual e o insere na posição correta desse prefixo, deslocando (e não trocando repetidamente) os elementos maiores uma posição à direita. Assim, ao final do processo, toda a sequência está em ordem.

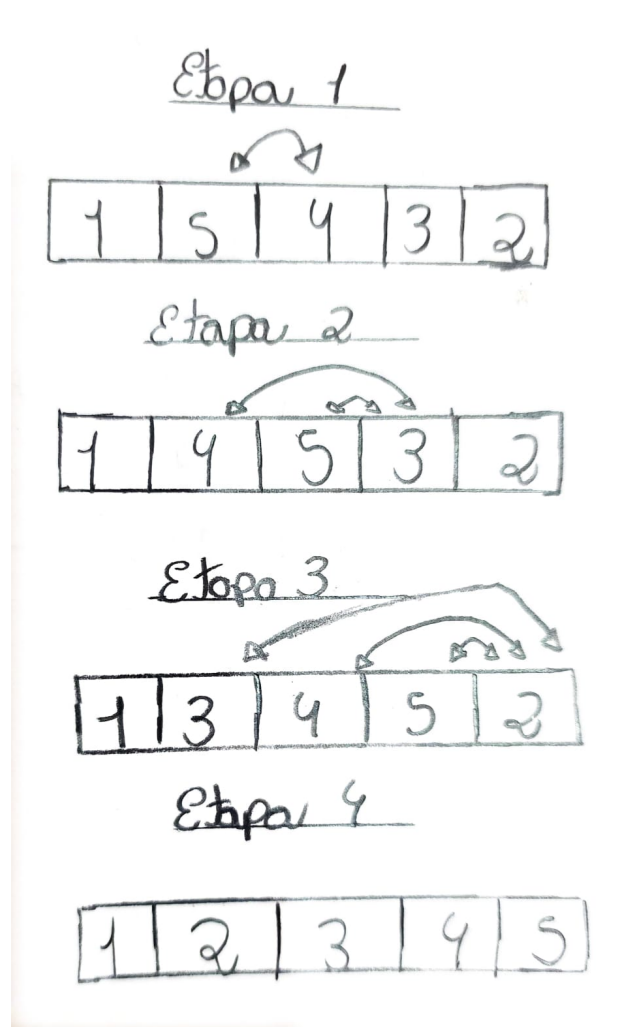


Figura 1: Insertion Sort funcionamento

Fonte: Autor

De acordo com a Figura 1, o algoritmo funciona da seguinte forma:

- **Etapa 1:** compara o número 4 com o 5. Como 4 é menor, ocorre a troca de posição.
- **Etapa 2:** o algoritmo segue para o 3. Como 3 é menor que 5, ocorre a troca de posição; logo depois, como 3 é menor que 4, ocorre nova troca.
- **Etapa 3:** compara o 2 com os elementos anteriores (5, depois 4 e depois 3), inserindo-o na posição correta no índice 1 do vetor.
- **Etapa 4:** terminando as comparações, o vetor se encontra ordenado.

A seguir teste de mesa.

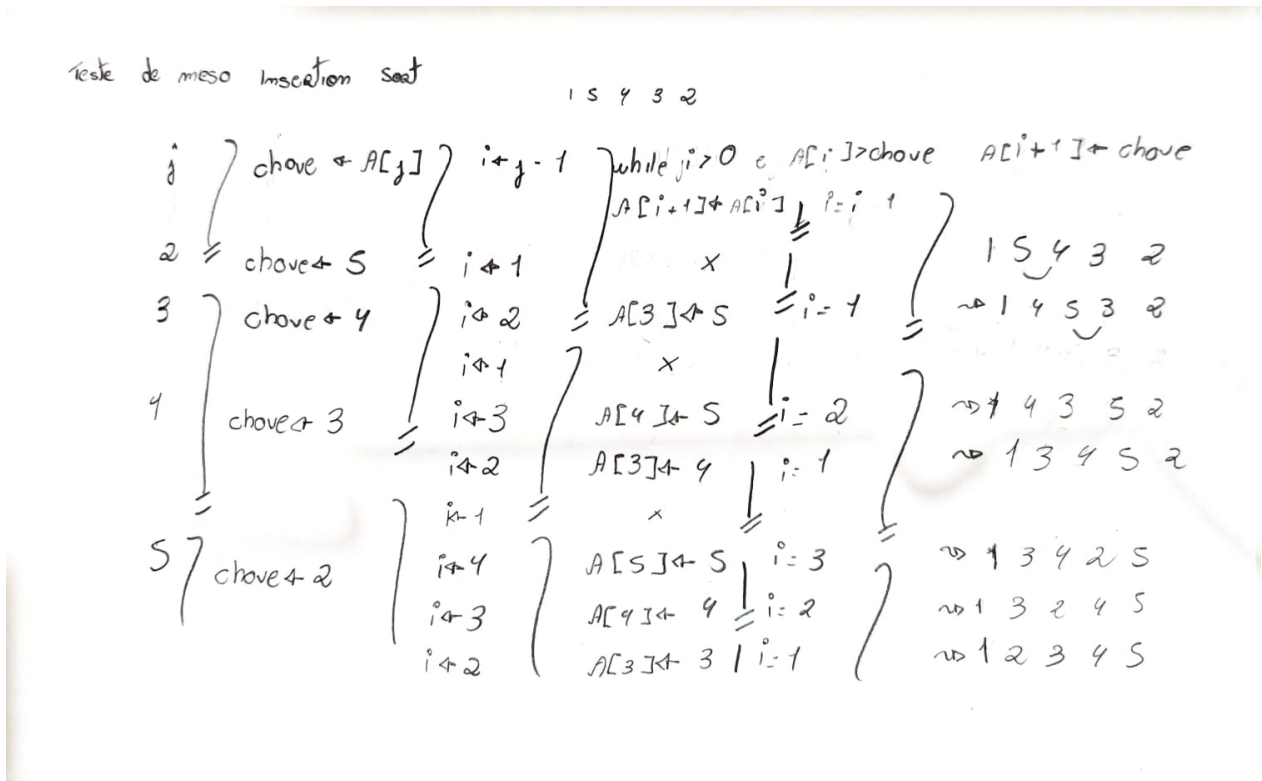


Figura 2: Teste de Mesa do Insertion Sort

Fonte: Autor

3.2 Selection Sort

O Selection Sort é um algoritmo de ordenação simples, cuja ideia central consiste em identificar repetidamente o menor (ou maior) elemento da parte não ordenada do vetor e posicioná-lo na sequência correta, deslocando gradualmente a fronteira entre os elementos ordenados e não ordenados. Esse procedimento de busca e troca é repetido até que todos os elementos estejam organizados. Apesar de realizar um número elevado de comparações em listas extensas, sua lógica direta e implementação enxuta tornam o algoritmo útil em aplicações onde a simplicidade é priorizada em relação ao desempenho.

De forma mais técnica, o Selection Sort divide o arranjo em duas partes: uma porção ordenada, que cresce progressivamente, e uma porção não ordenada, que diminui a cada iteração. A cada passo, o menor elemento da subsequência não ordenada é identificado e

trocado com o primeiro elemento dessa subsequência, fixando-o em sua posição definitiva. O processo continua até que todos os elementos estejam posicionados corretamente, resultando em uma ordenação completa da lista.

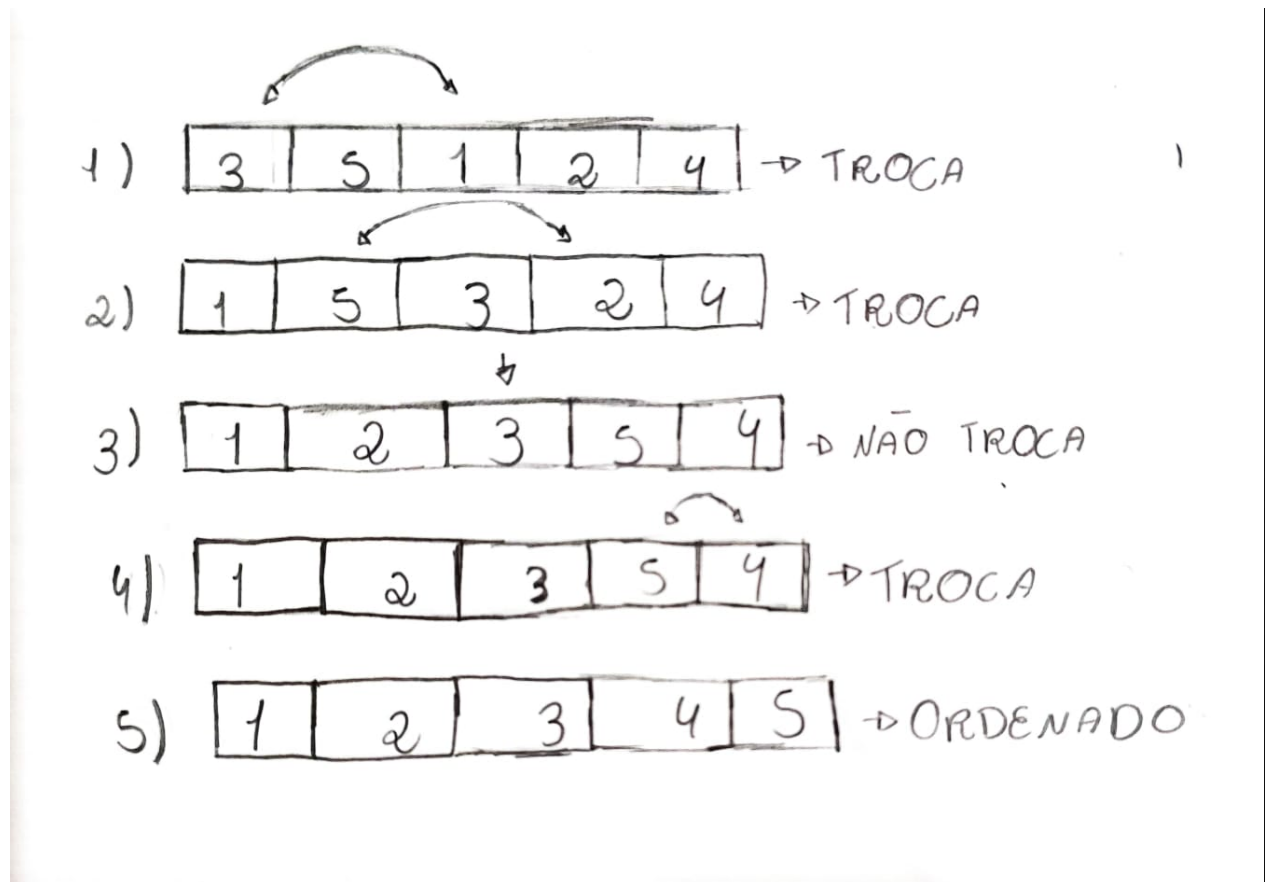


Figura 3: Funcionamento Selection Sort

Fonte: DevMedia

De acordo com a Figura 3, o algoritmo funciona da seguinte maneira:

- **Etapa 1:** o algoritmo procura o menor elemento de todo o vetor [3, 5, 1, 2, 4]. O menor é o 1, que troca de posição com o primeiro elemento. Resultado: [1, 5, 3, 2, 4].
- **Etapa 2:** no subvetor restante [5, 3, 2, 4], o menor é o 2, que troca de posição com o 5. Resultado: [1, 2, 3, 5, 4].
- **Etapa 3:** no subvetor [3, 5, 4], o menor é o 3, que já está na posição correta. Resultado: [1, 2, 3, 5, 4].

- **Etapa 4:** no subvetor final $[5, 4]$, o menor é o **4**, que troca de posição com o 5. Resultado: $[1, 2, 3, 4, 5]$.
- **Etapa 5:** ao final das comparações, o vetor encontra-se ordenado: $[1, 2, 3, 4, 5]$.

O teste de mesa é demonstrado a seguir.

Durante a execução do algoritmo *Selection Sort*, a comparação fundamental realizada em cada iteração é dada pela expressão $A[j] < A[\text{min}]$, em que $A[j]$ representa o elemento atual examinado no laço interno e $A[\text{min}]$ corresponde ao menor elemento encontrado até o momento. Sempre que a condição é verdadeira, ou seja, quando $A[j]$ é menor que $A[\text{min}]$, o índice do menor elemento é atualizado, passando a assumir o valor de j . Ao final do laço interno, realiza-se a troca entre $A[i]$ e $A[\text{min}]$, posicionando o menor elemento encontrado na posição correta do vetor.

Teste de mesa - selection sort

i	j	comparação	resultado	vetor
3	1			3 1 2 4
1	2	$A[2](1) < A[1](3)$	F	
1	3	$A[3](1) < A[1](3)$	V*min=3	
1	4	$A[4](2) < A[3](1)$	F	
1	5	$A[5](4) < A[3](1)$	F	
TROCA $A[1] \leftrightarrow A[3]$				1 3 2 4

Passo 1

i	j	comparação	resultado	vetor
2	3	$A3 < A2$	V*min=3	
2	4	$A[4](2) < A3$	V*min=4	
2	5	$A[5](4) < A[4](2)$	F	
TROCA $A[2] \leftrightarrow A[4]$				1 2 3 4

Passo 2

i	j	comparação	resultado	vetor
3	4	$A[4](5) < A3$	F	
3	5	$A[5](4) < A3$	F	
SEM TROCA				

Passo 3

i	j	comparação	resultado	vetor
4	5	$A[5](4) < A[4](5)$	V*min=5	
TROCA $A[4] \leftrightarrow A[5]$				1 2 3 4 5

Passo 4

i	j	comparação	resultado	vetor
5	-	-	-	-

Passo 5

Vetor final: 1 2 3 4 5

Figura 4: Teste de Mesa do Selection Sort

Fonte: Autor

3.3 Bubble Sort

O Bubble Sort é um algoritmo de ordenação simples que organiza os elementos por meio de comparações sucessivas entre pares adjacentes. Sempre que dois elementos estão fora de ordem, eles são trocados, fazendo com que, a cada passagem, o maior valor ainda não ordenado seja levado para o final do vetor. O processo se repete até que todos os elementos estejam corretamente posicionados.

Do ponto de vista técnico, o Bubble Sort executa diversas varreduras sobre a lista, comparando e, se necessário, trocando elementos adjacentes. Esse procedimento garante que, ao final de cada iteração, um dos maiores valores restantes esteja corretamente posicionado no final do arranjo. A repetição desse processo sucessivamente resulta em uma lista totalmente

ordenada.

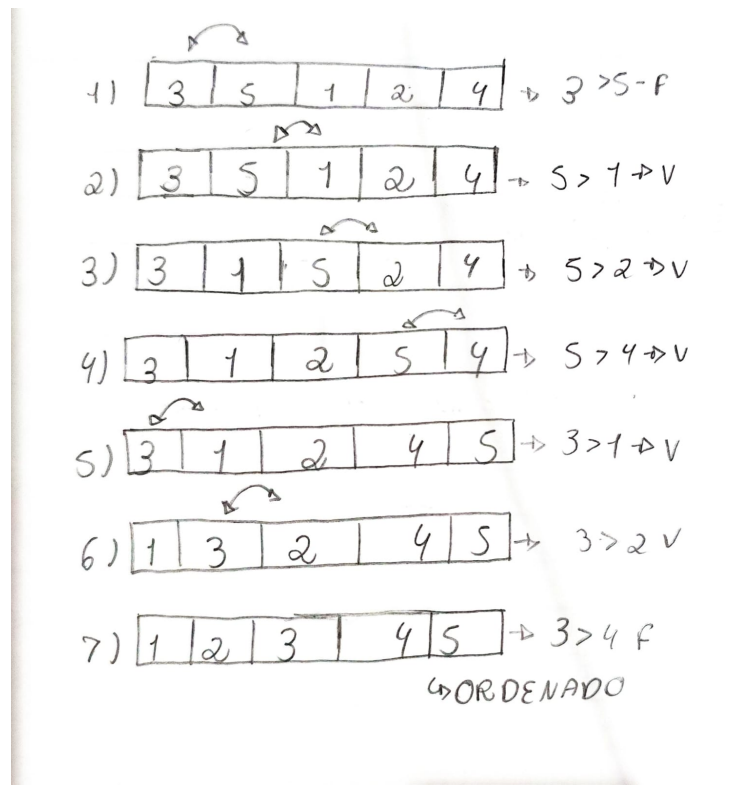


Figura 5: Funcionamento Bubble Sort

Fonte: DevMedia

- **Etapa 1:** compara-se 3 e 5. Como $3 < 5$, não ocorre troca. Vetor: [3, 5, 1, 2, 4].
- **Etapa 2:** compara-se 5 e 1. Como $5 > 1$, ocorre a troca. Vetor: [3, 1, 5, 2, 4].
- **Etapa 3:** compara-se 5 e 2. Como $5 > 2$, ocorre a troca. Vetor: [3, 1, 2, 5, 4].
- **Etapa 4:** compara-se 5 e 4. Como $5 > 4$, ocorre a troca. Vetor: [3, 1, 2, 4, 5].
- **Etapa 5:** compara-se 3 e 1. Como $3 > 1$, ocorre a troca. Vetor: [1, 3, 2, 4, 5].
- **Etapa 6:** compara-se 3 e 2. Como $3 > 2$, ocorre a troca. Vetor: [1, 2, 3, 4, 5].
- **Etapa 7:** compara-se 3 e 4. Como $3 < 4$, não ocorre troca. Vetor permanece: [1, 2, 3, 4, 5].
- **Resultado Final:** após essas comparações, o vetor está totalmente ordenado: [1, 2, 3, 4, 5].

Na sequência, tem-se a execução passo a passo do algoritmo.

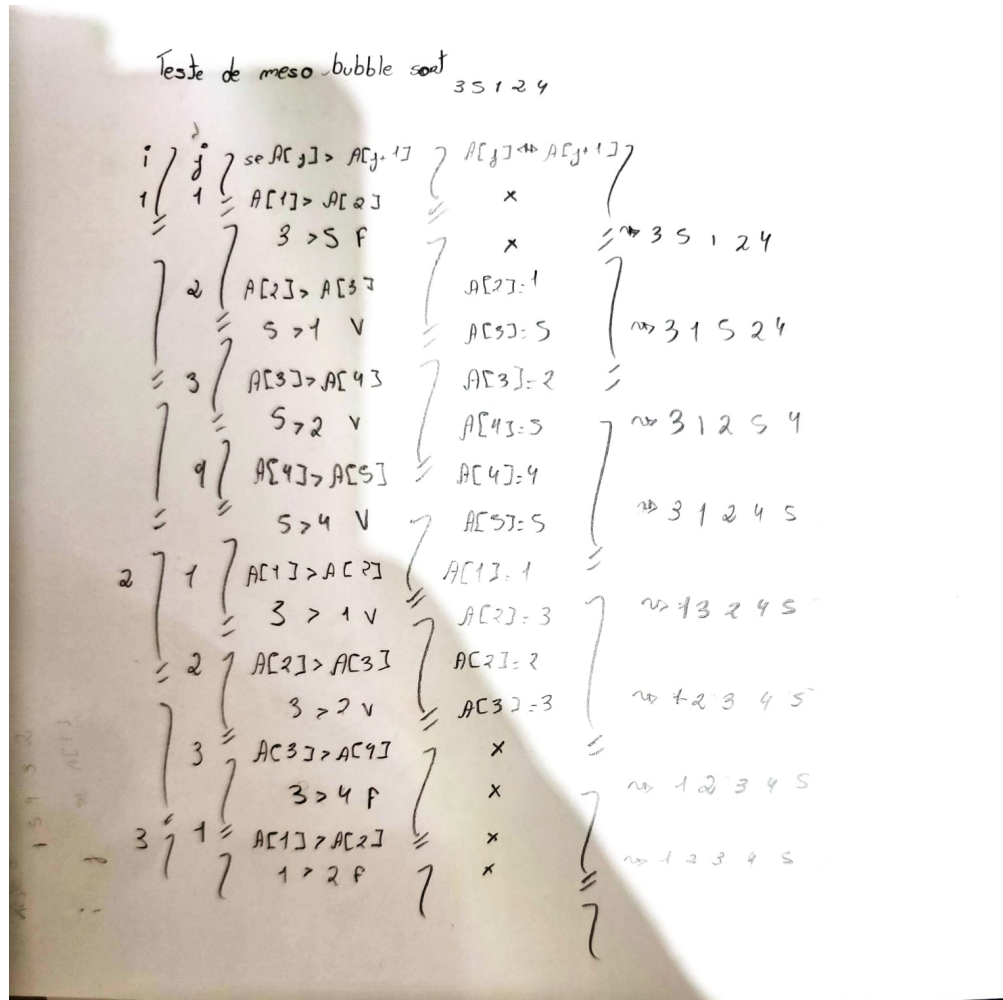


Figura 6: Teste de Mesa do Bubble Sort

Fonte: Autor

3.4 Shell Sort

O Shell Sort é um algoritmo de ordenação baseado no Insertion Sort, mas que melhora seu desempenho ao comparar e deslocar elementos que estão separados por um certo intervalo, chamado de *gap*. A cada iteração esse intervalo é reduzido até que se torne igual a 1, quando o algoritmo finaliza a ordenação de forma semelhante ao Insertion Sort.

Esse método permite que elementos distantes sejam movidos mais rapidamente para suas posições corretas, diminuindo o número total de deslocamentos necessários. Embora não seja

o mais eficiente entre os algoritmos de ordenação, o Shell Sort apresenta desempenho superior ao Insertion Sort em listas maiores e sua implementação continua relativamente simples.

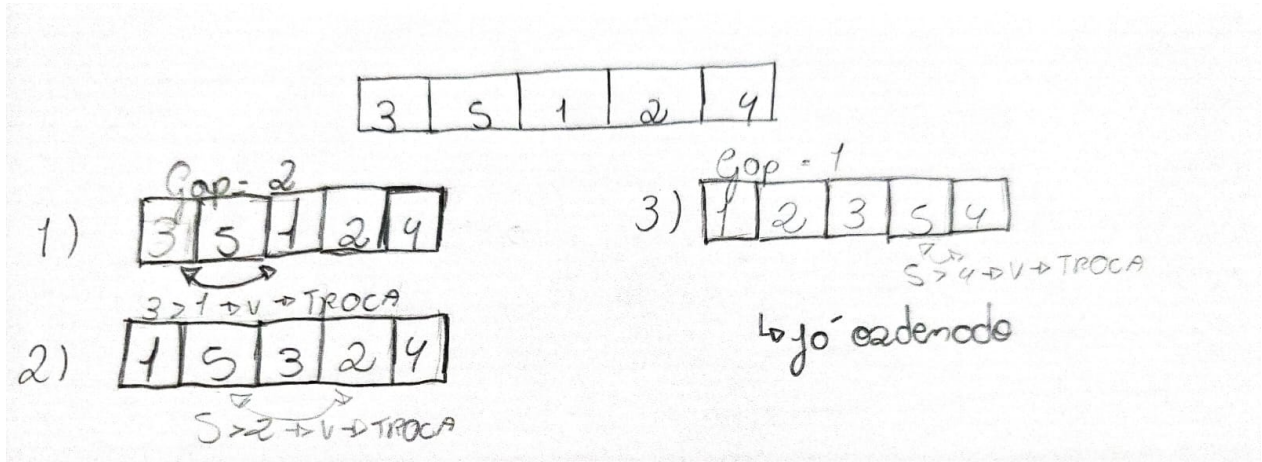


Figura 7: Funcionamento Shell Sort

Fonte: Autor

- **Etapa 1:** com $gap = 2$, compara-se 3 e 1. Como $3 > 1$, ocorre a troca. Vetor: [1, 5, 3, 2, 4].
- **Etapa 2:** ainda com $gap = 2$, compara-se 5 e 2. Como $5 > 2$, ocorre a troca. Vetor: [1, 2, 3, 5, 4].
- **Etapa 3:** com $gap = 1$, o algoritmo passa a funcionar como Insertion Sort. Compara-se 5 e 4. Como $5 > 4$, ocorre a troca. Vetor: [1, 2, 3, 4, 5].
- **Resultado Final:** após essas etapas, o vetor encontra-se totalmente ordenado: [1, 2, 3, 4, 5].

O funcionamento do algoritmo é verificado a seguir por meio do teste de mesa.

No algoritmo *Shell Sort*, a comparação central ocorre entre elementos separados por um intervalo denominado *gap*, que diminui progressivamente a cada iteração. Em cada subsequência formada, aplica-se um processo análogo ao *Insertion Sort*, comparando-se elementos segundo a relação $A[j] < A[j - gap]$. Caso a condição seja verdadeira, ou seja, se o elemento atual $A[j]$ for menor que o elemento situado à distância do intervalo ($A[j - gap]$),

realiza-se a troca entre essas posições, deslocando o menor valor para a parte anterior da sub-sequência. Esse procedimento repete-se para todos os elementos e para todos os valores de *gap*, até que este se torne igual a 1, momento em que o vetor é ordenado de forma completa.

teste de mesa shell sort $\rightarrow$ 3 5 1 2 4

$gap = 2 \rightarrow$ 1ª sequência $\rightarrow$ posições 1, 3, 5

i	j	comparação	resultado	vetor
1	-	-	-	3 5 1 2 4
3	1	$A[3](1) < A[1](3)$	V $\rightarrow$ TROCA	1 5 3 2 4
5	3	$A[5](4) < A3$	F	1 5 3 2 4

$\rightarrow$ Porção: 1 5 3 2 4

$\rightarrow$ 2ª sequência $\rightarrow$ posições 2, 4

i	j	comparação	resultado	vetor
2	-	-	-	1 5 3 2 4
4	2	$A[4](2) < A[2](5)$	V $\rightarrow$ TROCA	1 2 3 5 4

$gap = 1$

Aplicação final

i	j	comparação	resultado	vetor
2	1	$2 < 1$	F	
3	2	$3 < 2$	F	
4	3	$5 < 3$	F	
5	4	$4 < 5$	V $\rightarrow$ TROCA	1 2 3 4 5

Figura 8: Teste de Mesa do Shell Sort

Fonte: Autor

3.5 Merge Sort

O *Merge Sort* é um algoritmo de ordenação que organiza os elementos de uma lista de forma eficiente, combinando partes menores já ordenadas para formar uma sequência completa. Ele trabalha de maneira estruturada, reorganizando os dados progressivamente até que todos os elementos estejam em ordem, seja crescente ou decrescente.

Por manter um desempenho constante mesmo em listas grandes, o *Merge Sort* é considerado um dos métodos mais eficientes de ordenação. Além disso, apresenta resultados estáveis e previsíveis, sendo amplamente estudado e utilizado em livros e cursos sobre algoritmos.

O funcionamento do *Merge Sort* pode ser entendido em três etapas principais: divisão, conquista e combinação.

A etapa de divisão consiste em separar a sequência original em duas partes menores, cada uma com aproximadamente metade dos elementos. Essas partes formam duas subsequências que deverão ser ordenadas individualmente (CORMEN et al., 2012).

Na etapa de conquista, cada uma dessas subsequências é ordenada de forma recursiva, aplicando o mesmo processo de divisão até que se tornem listas simples e fáceis de organizar (CORMEN et al., 2012).

Por fim, na etapa de combinação, as subsequências já ordenadas são intercaladas, reunindo os elementos em uma única sequência de tamanho n totalmente ordenada (CORMEN et al., 2012).

A seguir, veja o funcionamento do Merge Sort:

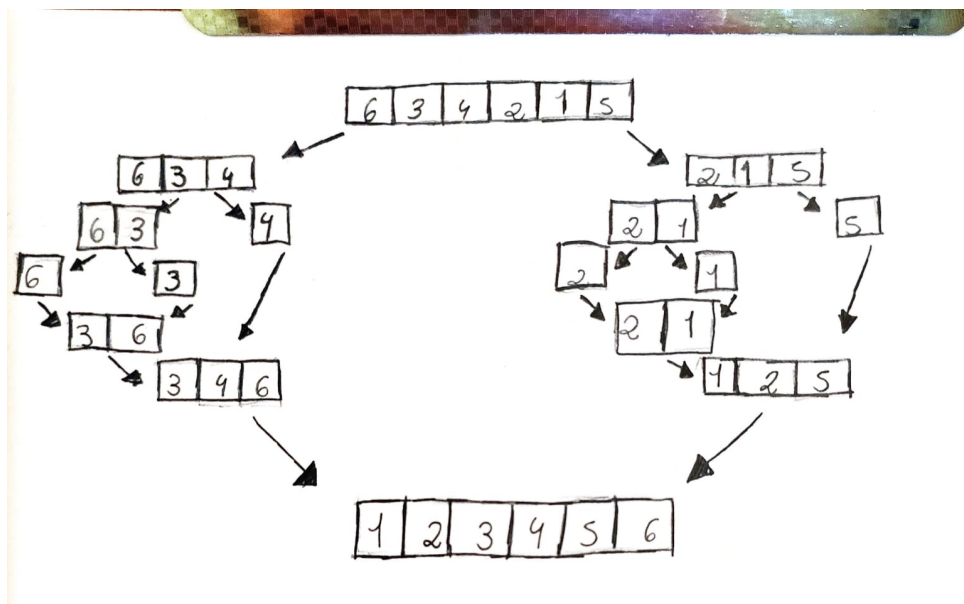


Figura 9: Funcionamento do Merge Sort

Fonte: Autor

Explicação:

Passo 1: o vetor é dividido sucessivamente ao meio até que cada parte contenha apenas um elemento, processando-se primeiro a metade esquerda e, em seguida, a direita.

Passo 2: cada subsequência é ordenada de forma recursiva por meio do próprio método Merge, garantindo que as partes sejam corretamente organizadas antes da combinação final.

Passo 3: as subsequências resultantes são intercaladas e reunidas em uma única sequência ordenada, formando o vetor final completamente organizado.

Segue a seguir o teste de mesa.

6 3 4 2 1 5							
i	j	$i < 0 + 1$ e $j < R + 1$	$if (v[i] < v[j])$	$thom$	$else$	w	
0	3	V	V	$3 < 1$ f	-	$w[0] = v[3] = 1$	[1]
0	4	V	V	$3 < 2$ f	-	$w[0] = v[4] = 2$	[1, 2]
0	5	V	V	$3 < 5$ V	$w[0] = 3$	-	[1, 3, 3]
1	5	V	V	$4 < 5$ V	$w[1] = 4$	-	[1, 2, 3, 4]
2	5	V	V	$6 < 5$ f	-	$w[2] = v[5] = 5$	[1, 2, 3, 4, 5]
2	6	V	f	-	-	-	[1, 2, 3, 4, 5, 6]

Figura 10: Teste de Mesa Merge Sort

Fonte: Autor

3.6 Quick Sort

O *Quick Sort* é um algoritmo que segue os padrões da *divisão e conquista*, porém não possui a etapa de combinação, pois cada elemento é colocado em sua posição ordenada após as divisões. Ele se mostra um algoritmo eficiente, pois divide o vetor em subvetores e os ordena de forma recursiva. O *Quick Sort* realiza sua ordenação por meio de divisões baseadas em um elemento chamado *pivô*, cujo objetivo é posicionar todos os elementos menores que ele à sua esquerda e os maiores à direita, sendo posteriormente colocado após os menores elementos. Por manter uma complexidade assintótica de $O(n \log n)$ no caso médio, o *Quick Sort* torna-se uma opção viável na maioria das situações práticas, visto que, na ausência de um estudo detalhado sobre a distribuição dos dados, o caso médio é o mais comum.

Vale ressaltar o funcionamento do *particiona*, uma das etapas do *Quick Sort*, que é responsável por escolher um *pivô* e, em seguida, reorganizar os elementos do vetor conforme descrito anteriormente, posicionando os menores à esquerda e os maiores à direita do *pivô*. No estudo em questão foi utilizado três diferentes escolhas do pivô, a primeira ele sendo o primeiro elemento, a segunda a média do primeiro e último elemento e a terceira de forma aleatória.

Segue adiante o funcionamento do Quick Sort o pivo sendo o ultimo elemento:

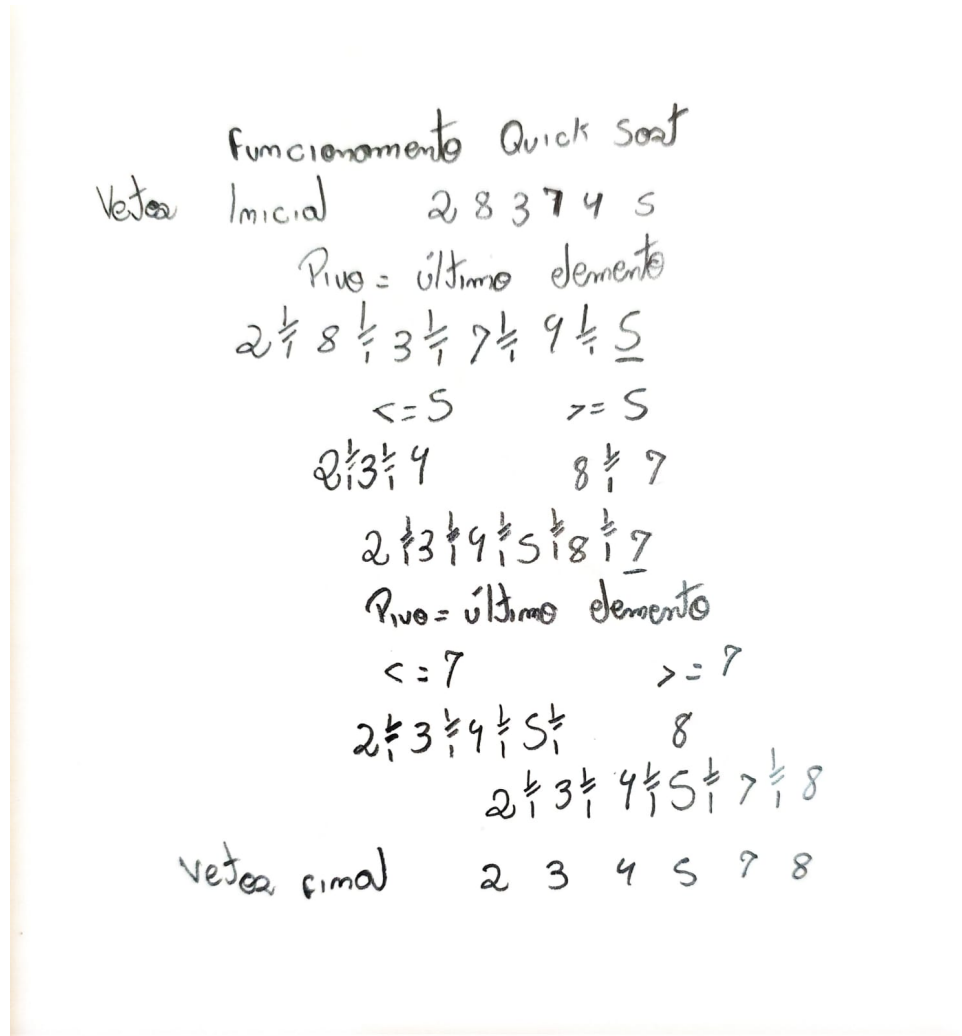


Figura 11: Funcionamento Quick Sort

Fonte: Autor

A seguir o teste de mesa do Quick Sort:

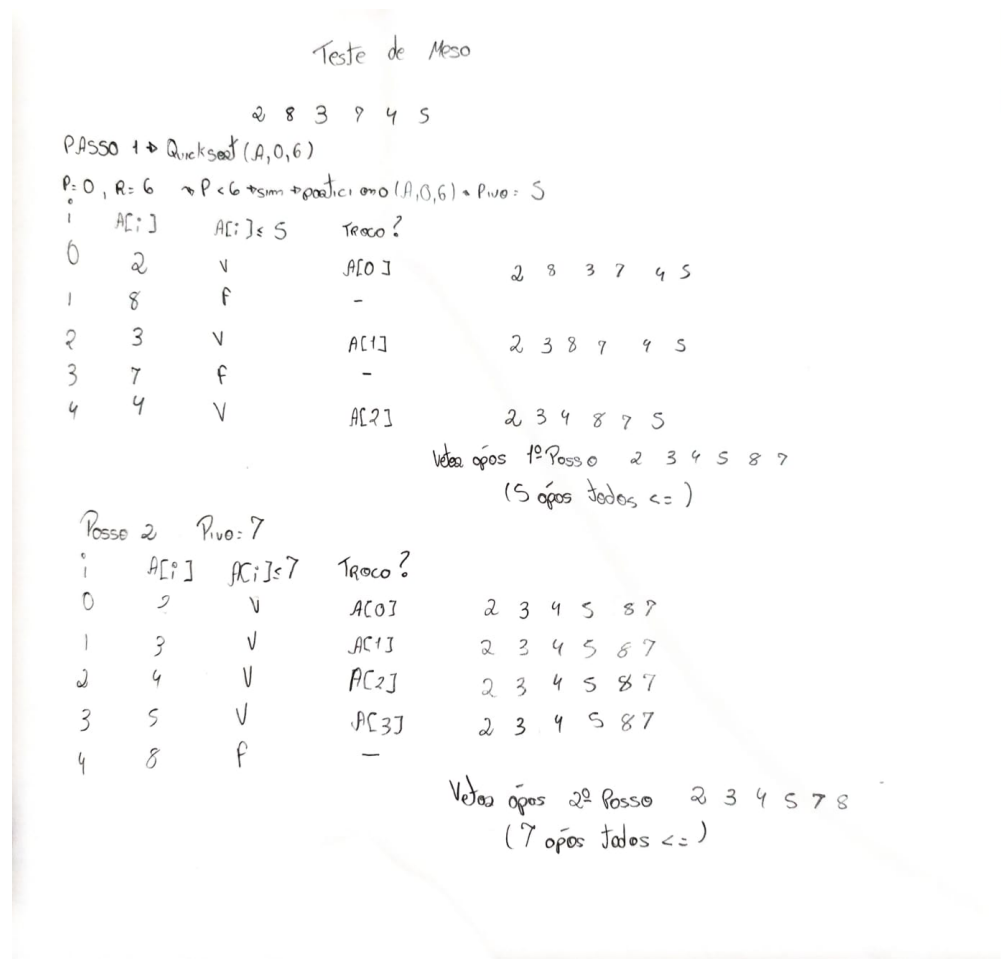


Figura 12: Teste de Mesa Quick Sort

Fonte: Autor

3.7 Heap Sort

O algoritmo *Heap Sort* baseia-se na estrutura de dados *heap*, utilizando suas propriedades para organizar os elementos de forma eficiente. O processo de construção e manutenção do *heap* é fundamental para o funcionamento e o desempenho do método.

3.7.1 Heap

A estrutura de dados *heap* é representada por um arranjo que pode ser interpretado como uma árvore binária quase completa. O tamanho dessa árvore é determinado pelo próprio arranjo, uma vez que cada nó corresponde a um elemento do vetor. Ressalta-se que a árvore

No *heap*, o elemento da raiz ocupa posição de destaque: em um *max-heap*, o valor armazenado na raiz é o maior do arranjo, sendo que cada nó possui valor menor ou igual ao de seu pai; já em um *min-heap*, ocorre o inverso a raiz contém o menor valor, e cada nó possui valor maior ou igual ao de seu pai. A seguir o funcionamento do Heap Sort:



Além do funcionamento apresentado, a seguir é exibido o teste de mesa que ilustra, passo a passo, o processo de ordenação realizado pelo algoritmo *Heap Sort*.

Teste de Mesa Heap Sort			
i	x	ocôe / Troco	Vetor
-	12	Troco 12 ↔ 4	[4, 5, 8, 3, 12]
0	4	Mover filho = 8 → Troco 8 ↔ 4	[8, 5, 4, 3, 12]
-	8	Troco 8 ↔ 3	[3, 5, 4, 8, 12]
0	3	Mover filho = 5 → Troco 3 ↔ 5	[5, 3, 4, 8, 12]
-	5	Troco 5 ↔ 4	[4, 3, 5, 8, 12]
0	4	Mover filho = 3 → Troco 4 ↔ 3	[4, 3, 5, 8, 12]
-	4	Troco 4 ↔ 3	[3, 4, 5, 8, 12]
0	-	Arranjo tamanho = 1 não há mais Troco vet final [3, 4, 5, 8, 12]	

Figura 14: Teste de Mesa Heap Sort

Fonte: Autor

O algoritmo *Heap Sort* realiza a ordenação a partir de sucessivas trocas entre o primeiro e o último elemento do arranjo. Após a construção do *heap* (seja ele *max-heap* ou *min-heap*), o elemento da raiz que representa o maior ou o menor valor, conforme o tipo de *heap* é trocado com o último elemento do vetor. Em seguida, o tamanho efetivo do *heap* é reduzido, desconsiderando a posição já ordenada, e a propriedade de *heap* é restaurada. Esse processo é repetido até que todos os elementos estejam em suas posições corretas, resultando em um vetor totalmente ordenado.

3.7.2 Filas de Prioridade

As filas de prioridade estão diretamente relacionadas à estrutura de dados *heap*, utilizada no algoritmo *Heap Sort*. Uma fila de prioridade é uma abstração que permite o acesso rápido ao

elemento de maior ou menor prioridade dentro de um conjunto de dados, dependendo do tipo de *heap* empregado. Em um *max-heap*, o elemento de maior valor possui a maior prioridade e é armazenado na raiz; já em um *min-heap*, ocorre o inverso. As operações fundamentais de inserção e remoção em filas de prioridade exploram as propriedades do *heap*, garantindo complexidade $O(\log n)$ por operação. Essa relação torna o *heap* uma das implementações mais eficientes para o gerenciamento dinâmico de prioridades em aplicações computacionais.

As principais operações realizadas em filas de prioridade implementadas com *heap* são: *insert*, que insere um novo elemento mantendo a propriedade de *heap*; *maximum*, que retorna o elemento de maior valor (no caso de um *max-heap*); *extract-max*, que remove e retorna o maior elemento, reorganizando a estrutura; e *increase-key*, que ajusta o valor de uma chave já existente, garantindo a consistência da hierarquia. Tais operações possuem complexidade $O(\log n)$ e são amplamente discutidas em (CORMEN 2012).

4 ANÁLISE DE COMPLEXIDADE

Nesta seção, serão discutidas as análises de complexidade dos algoritmos, entendidas como medidas que avaliam o tempo de execução em função do tamanho da entrada. Essa análise é fundamental para a escolha adequada do algoritmo, garantindo o melhor aproveitamento possível dos recursos computacionais.

A partir deste ponto, será apresentada a complexidade dos algoritmos estudados, destacando os diferentes cenários de execução e indicando em quais situações seu uso se mostra mais vantajoso.

4.1 Insertion Sort

Observa-se que N corresponde ao comprimento do vetor A .

Denota-se por T_j o número de vezes que o teste do laço while na linha 5 é executado para um determinado valor de j .

Importa destacar que, quando um laço for ou while termina de forma usual — isto é, devido ao teste presente em seu cabeçalho —, esse teste é realizado uma vez a mais além das execuções do corpo do laço. Adiante o algoritmo:

Insertion Sort		Custo	Vezez
1-	para $j \leftarrow$ to comprimento	C_1	N
2-	do $chave \leftarrow A[j]$	C_2	$(N-1)$
3-	"insere $A[j]$ na seq ordenada $A[1, \dots, j-1]$	C_3	$(N-1)$
4-	$i \leftarrow j - 1$	C_4	$(N-1)$
5-	while $i > 0$ e $A[i] > chave$	C_5	$\sum_{j=2}^N T_j$
6-	do $A[i+1] \leftarrow A[i]$	C_6	$\sum_{j=2}^N T_j - 1$
7-	$i \leftarrow i - 1$	C_7	$\sum_{j=2}^N T_j - 1$
8-	$A[i+1] \leftarrow chave$	C_8	$(N-1)$

Figura 15: Algoritmo Insertion Sort

Fonte: Autor

De acordo com Figura 15, pode-se observar o funcionamento do algoritmo.

	CUSTO	VEZES
1	C1	N
2	C2	N-1
3	C3	N-1
4	C4	N-1
5	C5	$\sum_{j=2}^n T_j$
6	C6	$\sum_{j=2}^n T_j - 1$
7	C7	$\sum_{j=2}^n T_j - 1$
8	C8	N-1

Tabela 1: Custo InsertionSort

Fonte: Autor

De acordo com a Tabela 1, observam-se de forma clara os custos que serão utilizados nos cálculos a seguir.

4.1.1 Forma Geral

De forma geral, os valores de custo e frequência apresentados na Tabela 1 permitem observar o comportamento do algoritmo.

A formulação geral correspondente será apresentada a seguir.

$$J(n) = C1(n) + C2(n-1) + C3(n-1) + C4(n-1) + C5\left(\sum_{j=2}^n T_j\right) + C6\left(\sum_{j=2}^n T_j - 1\right) + C7\left(\sum_{j=2}^n T_j - 1\right) + C8(n-1)$$

Figura 16: Fórmula Geral Insertion Sort

Fonte: Autor

4.1.2 Melhor Caso

O melhor caso no Insertion Sort ocorre quando a condição da linha 5 ($A[i] > chave$) é sempre falsa. Nessa situação, as linhas 6 e 7 não são executadas, pois não há necessidade de deslocar elementos.

O cálculo do melhor caso é apresentado a seguir.

$$\begin{aligned}f(m) &= C_1(m) + C_2(m-1) + C_3(m-1) + C_4(m-1) + C_5(m-1) + C_8(m-1) \\f(m) &= (C_1 + C_2 + C_3 + C_4 + C_5 + C_8)m + (-C_2 - C_3 - C_4 - C_5 - C_8) \\f(m) &= A'm + B'\end{aligned}$$

Figura 17: Melhor Caso Insertion Sort

Fonte: Autor

Como visto no resultado final, a função $T(n) = A'n + B'$ é linear. Portanto, a complexidade de tempo no melhor caso é $\Omega(n)$.

4.1.3 Pior Caso

O pior caso do Insertion Sort ocorre quando o arranjo está ordenado em ordem inversa. Nessa situação, cada elemento $A[j]$ precisa ser comparado com todos os elementos do subarranjo anterior, resultando no maior número possível de comparações e deslocamentos.

O cálculo do pior caso é apresentado a seguir.

$$\begin{aligned}
\sum_{j=2}^N T_j &= \sum_{j=2}^N j = \frac{(n-1)(2+n)}{2} \rightarrow \frac{2n - 2 + n^2 - n}{2} \rightarrow \frac{n^2 + n - 2}{2} \\
\sum_{j=2}^N T_j - 1 &= \sum_{j=2}^N j - 1 = \sum_{j=2}^N j - \sum_{j=2}^N 1 \\
&\rightarrow \left[\frac{n^2 + n - 2}{2} \right] - [n-1] \rightarrow \frac{n^2 + n - 2 - 2[n-1]}{2} \rightarrow \frac{n^2 + n - 2 - 2n + 2}{2} \rightarrow \frac{n^2 - n}{2} \\
T(n) &= C_1 n + C_2(n-1) + C_3(n-1) + C_4(n-1) + C_5 \left(\frac{n^2 + n - 2}{2} \right) + C_6 \left(\frac{n^2 - n}{2} \right) + C_7 \left(\frac{n^2 - n}{2} \right) + C_8(n-1) \\
&\rightarrow C_1 n + C_2 n - C_2 + C_3 n - C_3 + C_4 n - C_4 + C_5 \frac{n^2}{2} + C_5 \frac{n}{2} - C_5 + C_6 \frac{n^2}{2} - C_6 \frac{n}{2} + C_7 \frac{n^2}{2} + C_7 \frac{n}{2} + C_8 n - C_8 \\
&\rightarrow \frac{(C_5 + C_6 + C_7)n^2}{2} + (C_1 + C_2 + C_3 + C_4 + \frac{C_5}{2} + \frac{C_6}{2} - \frac{C_7}{2} + C_8)n + (-C_2 - C_3 - C_4 - C_5 - C_8) \\
&\rightarrow T(n) = A'n^2 + B'n + C'
\end{aligned}$$

Figura 18: Pior caso Insertion Sort

Fonte: Autor

Como visto nos cálculos da Figura 18, a função $T(n) = A'n^2 + B'n + C'$ apresenta crescimento quadrático. Portanto, a complexidade de tempo no pior caso é $\mathcal{O}(n^2)$.

4.1.4 Medio Caso

No caso médio, a forma da função mantém-se quadrática, havendo apenas variação nos coeficientes. Assim, a complexidade assintótica permanece em $\Theta(n^2)$, como demonstrado nos cálculos a seguir. Considera-se que, em média, $T_j = \frac{j}{2}$, pois cada elemento é comparado com aproximadamente metade do subarranjo já ordenado.

O cálculo do medio caso é apresentado a seguir.

$$\begin{aligned}
\sum_{j=2}^N j &= \sum_{j=2}^N \sum_{i=j}^N 1 \rightarrow \sum_{j=2}^N \frac{1}{2} j = \frac{1}{2} \sum_{j=2}^N j \\
\hookrightarrow \frac{1}{2} \left[\frac{(n-1)(2+n)}{2} \right] &= \frac{1}{2} \left(\frac{n^2+n-2}{2} \right) \rightarrow \frac{n^2+n-2}{4} \rightarrow \frac{n^2+n}{4} - \frac{1}{2} \\
\sum_{j=2}^N \frac{1}{2} - 1 &= \sum_{j=2}^N \frac{1}{2} - \sum_{j=2}^N 1 \rightarrow \frac{n^2+n-2}{4} - [n-1] = \frac{n^2+n-2-4[n-1]}{4} \rightarrow \frac{n^2+n-2-4n+4}{4} \\
&\rightarrow \frac{n^2-3n+2}{4} \rightarrow \frac{n^2-3n}{4} + \frac{1}{2} \\
T(n) &= C_1n + C_2(n-1) + C_3(n-1) + C_4(n-1) + C_5 \left(\sum_{j=2}^N j \right) + C_6 \left(\sum_{j=2}^N j - 1 \right) + C_7 \left(\sum_{j=2}^N \frac{1}{2} \right) + C_8(n-1) \\
T(n) &= C_1n + C_2(n-1) + C_3(n-1) + C_4(n-1) + C_5 \left(\frac{n^2+n}{4} - \frac{1}{2} \right) + C_6 \left(\frac{n^2-3n}{4} + \frac{1}{2} \right) + C_7 \left(\frac{n^2-3n}{4} + \frac{1}{2} \right) + C_8(n-1) \\
T(n) &= \left(\frac{C_5}{4} + \frac{C_6}{4} + \frac{C_7}{4} \right) n^2 + (C_1 + C_2 + C_3 + C_4 + \frac{C_5}{4} - \frac{3C_6}{4} - \frac{3C_7}{4} + C_8) n + \left(\frac{C_5}{2} - C_3 - C_4 - \frac{C_6}{2} - \frac{C_7}{2} - C_8 \right) \\
T(n) &= A'n^2 + B'n + C'
\end{aligned}$$

Figura 19: Caso Médio Insertion Sort

Fonte: Autor

Como demonstrado na Figura 19, a função $T(n) = A'n^2 + B'n + C$ apresenta crescimento quadrático, confirmando, portanto, a análise previamente realizada.

4.1.5 Conclusão da Análise do Insertion Sort

Como visto nos exemplos de melhor, médio e pior caso, nota-se que, para arranjos já ordenados, o Insertion Sort é bastante útil e eficiente. No entanto, quando aplicado a conjuntos de dados desordenados, o algoritmo torna-se inviável devido à sua tendência quadrática de crescimento.

4.2 Selection Sort

Observa-se que N corresponde ao comprimento do vetor A . No algoritmo Selection Sort, a cada iteração o menor elemento da parte não ordenada é identificado e colocado na posição correta, por meio de uma troca. Esse procedimento exige que o algoritmo percorra diversas

vezes os elementos ainda restantes do vetor em busca do menor valor.

A seguir o algoritmo:

SELECTION SORT		CUSTO	VEZES
1-	for (i = 0; i < tom - 1; i++) {	C1	N
2-	minIdx = i;	C2	N-1
3-	for (j = i+1; j < tom; j++) {	C3	$\sum_{j=2}^N T_j$
4-	if (v[j] < v[minIdx])	C4	$\sum_{j=2}^N T_j - 1$
5-	minIdx = j;	C5	$\sum_{j=2}^N T_j - 1$
6-	if (minIdx != i)	C6	N-1
7-	changeEl(&v[minIdx], &v[i]);		
8-	}		
9-	}		

Figura 20: Algoritmo Selection Sort

Fonte: Autor

4.2.1 Forma Geral

A seguir, apresenta-se a formulação geral correspondente.

$$T(m) = C1m + C2(m-1) + C3\left(\sum_{j=2}^N T_j\right) + C4\left(\sum_{j=2}^N T_j - 1\right) + C5\left(\sum_{j=2}^N T_j - 1\right) + C6(m-1)$$

Figura 21: Formula Geral Selection Sort

Fonte: Autor

4.2.2 Melhor Caso

O melhor caso no Selection Sort ocorre quando o menor elemento de cada subsequência não ordenada já se encontra na primeira posição dessa subsequência. Nessa situação, a linha responsável pela troca de elementos não é executada, pois não há necessidade de reposicionar valores — o algoritmo apenas percorre os elementos para confirmar a ordenação.

$$T(m) = C_1 m + C_2(m-1) + C_3 \left(\frac{m^2 + m - 2}{2} \right) + C_4(m^2 - m^2) + C_6(m-1)$$

$$T(m) = C_1 m + C_2 m - C_2 + \frac{C_3 m^2 + C_3 m - 2C_3}{2} + C_6 m - C_6$$

$$T(m) = (C_3)m^2 + (C_1 + C_2 + C_3 + C_6)m + (-C_2 - 2C_3 - C_6)$$

$$T(m) = A'm^2 + Bm + C$$

Figura 22: Melhor Caso

Fonte: Autor

Como visto no resultado final, a função $T(n) = A'n^2 + B'n + C'$ é quadrática. Portanto, a complexidade de tempo no melhor caso é $O(n^2)$.

4.2.3 Pior Caso

No Selection Sort, o pior caso ocorre quando o vetor está em ordem decrescente, pois em cada iteração o menor elemento da subsequência não ordenada encontra-se na última posição, exigindo a realização de uma troca a cada passo. Ainda assim, o número de comparações permanece igual ao do melhor caso, dado por $\frac{n(n-1)}{2}$, o que torna a função de custo quadrática. Portanto, a complexidade de tempo no pior caso é $O(n^2)$, coincidindo com o melhor caso, como demonstrado a seguir.

$$f(m) = C_1 m + C_2(m-1) + C_3 \left(\frac{m^2 + m - 2}{2} \right) + C_4(m^2 - m^2) + C_5(m^2 - m^2) + C_6(m-1)$$

$$f(m) = C_1 m + C_2 m - C_2 + \frac{C_3 m^2 + C_3 m - 2C_3}{2} + C_6 m - C_6$$

$$f(m) = (C_3)m^2 + (C_1 + C_2 + C_3 + C_6)m + (-C_2 - 2C_3 - C_6)$$

$$f(m) = A m^2 + B m + C$$

Figura 23: Pior Caso

Fonte: Autor

4.2.4 Médio Caso

No Selection Sort, o caso médio ocorre quando o vetor está em uma ordem aleatória, sem estar previamente ordenado ou totalmente invertido. Nessa situação, em cada iteração é necessário percorrer a subsequência não ordenada para identificar o menor elemento, realizando, em média, uma troca por passo. O número de comparações, contudo, permanece constante em relação aos demais cenários, sendo dado por $\frac{n(n-1)}{2}$. Dessa forma, a função de custo é quadrática e a complexidade de tempo no caso médio é $O(n^2)$, como será demonstrado a seguir.

$$f(m) = C_1 m + C_2(m-1) + C_3 \frac{m^2 + m - 1}{2} + C_4 \frac{1}{4} \left(\sum_{j=2}^m j(j-1) \right) + C_5 \frac{1}{4} \left(\sum_{j=2}^m j(j-1) \right) + C_6(m-1)$$

$$f(m) = (C_3) \frac{m^2}{2} + (C_1 + C_2 + C_3 + C_4 + C_5)m + (-C_2 - \frac{1}{2}(C_3 - C_4 - C_5 - C_6))$$

$$f(m) = A m^2 + B m + C$$

Figura 24: Médio Caso

Fonte: Autor

4.2.5 Conclusão da Análise do Selection Sort

Como visto nos exemplos de melhor, médio e pior caso, nota-se que, para o algoritmo Selection Sort, o custo de execução permanece o mesmo independentemente da organização inicial dos dados. Isso ocorre porque, a cada iteração, o algoritmo precisa percorrer todo o subarranjo restante para encontrar o menor elemento, resultando em um número fixo de comparações. Dessa forma, tanto no melhor quanto no pior e no médio caso, o Selection Sort apresenta um tempo de execução de ordem quadrática, ou seja, $O(n^2)$.

4.3 Bubble Sort

No algoritmo Bubble Sort, dado um vetor A de tamanho N , são realizadas diversas passagens completas pelo arranjo. Em cada passagem, os elementos vizinhos são comparados um a um, e sempre que estão fora de ordem ocorre a troca de posição.

A cada nova varredura, o maior valor ainda não ordenado é deslocado para o final da sequência, reduzindo gradualmente o número de comparações necessárias. O algoritmo pode ser descrito da seguinte forma:

Bubble Sort		Custo	Vezez
1 - para $i = 1$ até comprimento $[A] - 1$		$C1$	$N - 1$
2 - para $j = 1$ até comprimento $[A] - i$		$C2$	$\sum_{j=2}^{N-1} T_j$
3 - se $A[j] > A[j+1]$ então		$C3$	$\sum_{j=2}^{N-1} T_j - 1$
4 - troca $A[j]$ e $A[j+1]$		$C4$	$\sum_{j=2}^{N-1} T_j - 1$

Figura 25: Algoritmo Bubble Sort

Fonte: Autor

4.3.1 Forma Geral

No Bubble Sort, o total de comparações segue a fórmula geral.

$$T(n) = C_1(n-1) + C_2\left(\sum_{j=2}^{n-1} T_j\right) + C_3\left(\sum_{j=2}^{n-1} T_j - 1\right) + C_4\left(\sum_{j=2}^{n-1} T_j - 1\right)$$

Figura 26: Forma Geral Bubble Sort

Fonte: Autor

Segue adiante o desenvolvimento dos somatórios.

$$\begin{aligned}\sum_{j=1}^{n-1} T_j &= \sum_{j=1}^{n-1} j = \frac{(n-1)(1+n-1)}{2} = \frac{(n-1)(n)}{2} = \frac{n^2 - n}{2} \\ \sum_{j=1}^{n-1} T_j - 1 &= \sum_{j=1}^{n-1} j - \sum_{j=1}^{n-1} 1 = \frac{(n-1)(n)}{2} - [n-1] = \frac{n^2 - 3n + 2}{2}\end{aligned}$$

Figura 27: Somatório Bubble Sort

Fonte: Autor

4.3.2 Melhor Caso

O melhor caso no Bubble Sort ocorre quando o vetor já está ordenado em ordem crescente. Nessa situação, a condição da linha 3 ($A[j] > A[j+1]$) é sempre falsa. Assim, a linha 4 não é executada, pois não há necessidade de realizar trocas entre elementos.

Cálculo do melhor caso a seguir.

$$\begin{aligned}
 T(n) &= C_1(n-1) + C_2\left(\frac{n^2-n}{2}\right) + C_3\left(\frac{n^2-3n}{2} + \frac{1}{2}\right) + C_4(0) \\
 T(n) &= C_1n - C_1 + \frac{C_2n^2}{2} - \frac{C_2n}{2} + \frac{C_3n^2}{2} - \frac{3C_3n}{2} + \frac{C_3}{2} \\
 T(n) &= \left(\frac{C_2+C_3}{2}\right)n^2 + \left(C_1 - \frac{C_2}{2} - \frac{3C_3}{2}\right)n + \left(-C_1 + \frac{C_3}{2}\right) \\
 T(n) &= A'n^2 + B'n + C'
 \end{aligned}$$

Figura 28: Melhor Caso Bubble Sort

Fonte: Autor

Como visto no resultado final, a função $T(n) = A'n^2 + B'n + C'$ é quadrática. Portanto, a complexidade de tempo no melhor caso do Bubble Sort é $O(n^2)$.

4.3.3 Médio e Pior Caso

O pior caso do Bubble Sort ocorre quando o arranjo está em ordem inversa. Nessa situação, cada comparação entre elementos adjacentes resulta em troca, levando ao maior número possível de movimentações ao longo da execução.

O caso médio ocorre quando o arranjo está em ordem aleatória. Nessa condição, o algoritmo realiza todas as comparações previstas nos laços, porém, em média, aproximadamente metade dessas comparações gera uma troca. Assim, embora o número de trocas seja menor que no pior caso, o número de comparações segue a mesma fórmula geral, isto é, a soma de $n - 1$ até 1.

$$\begin{aligned}
 f(n) &= C_1(n-1) + C_2\left(\frac{n^2-n}{2}\right) + C_3\left(\frac{n^2-3n}{2} + \frac{1}{2}\right) + C_4\left(\frac{n^2-3n}{2} + \frac{1}{2}\right) \\
 f(n) &= C_1n - C_1 + \frac{C_2n^2}{2} - \frac{C_2n}{2} + \frac{C_3n^2}{2} - \frac{3C_3n}{2} + \frac{C_3}{2} + \frac{C_4n^2}{2} - \frac{3C_4n}{2} + \frac{C_4}{2} \\
 f(n) &= \left(\frac{C_2}{2} + \frac{C_3}{2} + \frac{C_4}{2}\right)n^2 + \left(C_1 - \frac{C_2}{2} - \frac{3C_3}{2} - \frac{3C_4}{2}\right)n + \left(-C_1 + \frac{C_3}{2} + \frac{C_4}{2}\right) \\
 f(n) &= A'n^2 + B'n + C'
 \end{aligned}$$

Figura 29: Pior e Médio Caso Bubble Sort

Fonte: Autor

Como visto nos cálculos apresentados, a função $T(n) = A'n^2 + B'n + C'$ apresenta crescimento quadrático. Portanto, a complexidade de tempo no médio e no pior caso do Bubble Sort é $O(n^2)$.

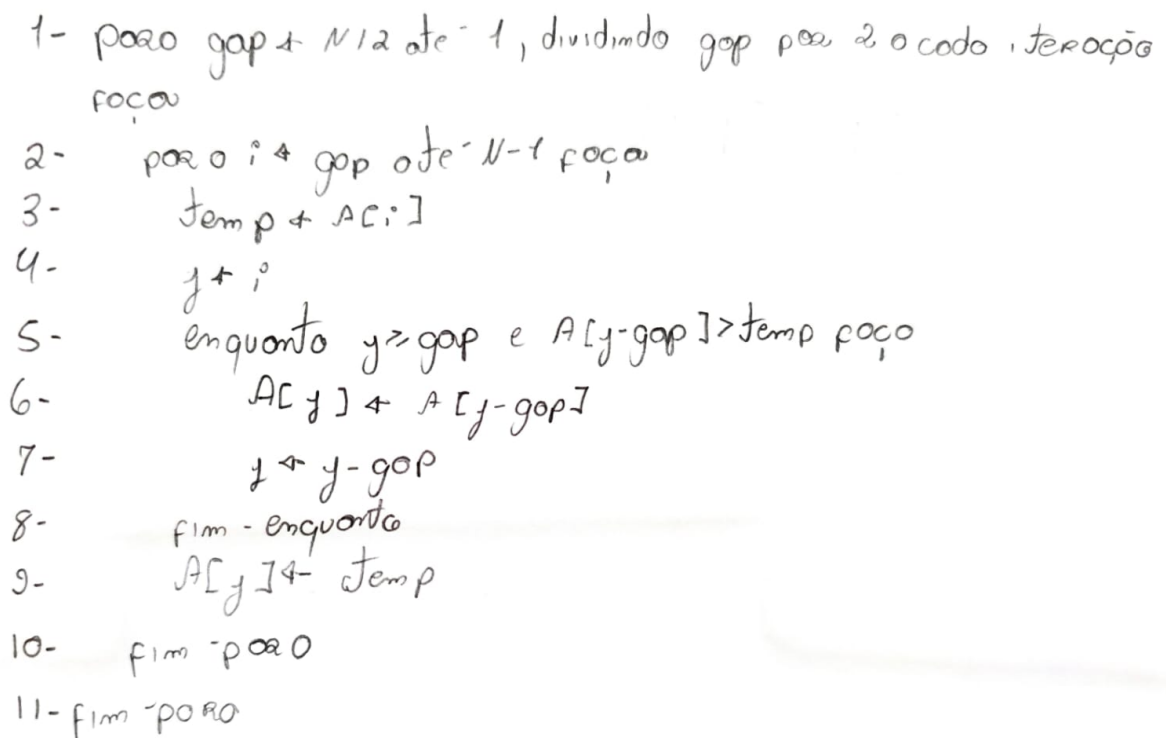
4.3.4 Conclusão da Análise do Bubble Sort

Como visto nos exemplos de melhor, médio e pior caso, nota-se que, para arranjos já ordenados, o Bubble Sort não apresenta ganho significativo, pois ainda executa todas as comparações previstas. No entanto, quando aplicado a conjuntos de dados de qualquer natureza, o algoritmo mostra-se inviável devido à sua tendência quadrática de crescimento, o que limita sua eficiência em vetores de tamanho elevado.

4.4 Shell Sort

Considerando um vetor A de tamanho N , observa-se que o algoritmo Shell Sort executa diversas fases de ordenação baseadas em um valor de incremento, chamado *gap*. Em cada fase, elementos separados por essa distância são comparados e, quando necessário, deslocados para novas posições.

Durante o processo, os testes de condição presentes nos laços são avaliados tanto nas comparações propriamente ditas entre os elementos quanto nas verificações que determinam o fim das repetições. Esse padrão se repete em cada etapa até que o *gap* seja reduzido a 1, quando a sequência final de comparações garante que o vetor esteja ordenado. A descrição do algoritmo está indicada a seguir:



```

1- para  $gap \leftarrow N/2$  até 1, dividindo  $gap$  por 2 a cada iteração
   faça
2-   para  $i \leftarrow gap$  até  $N-1$  faça
3-      $temp \leftarrow A[i]$ 
4-      $j \leftarrow i$ 
5-     enquanto  $j \geq gap$  e  $A[j-gap] > temp$  faça
6-        $A[j] \leftarrow A[j-gap]$ 
7-        $j \leftarrow j - gap$ 
8-     fim - enquanto
9-      $A[j] \leftarrow temp$ 
10-  fim - para
11- fim - para
  
```

Figura 30: Algoritmo Shell Sort

Fonte: Adaptado de Shell(1959)

4.4.1 Complexidade Shell Sort

Diferentemente de algoritmos básicos de ordenação, como o *Insertion Sort* ou o *Bubble Sort*, o *Shell Sort* não possui uma expressão fechada para a determinação exata de sua complexidade de tempo. Isso ocorre porque o desempenho do algoritmo está diretamente condicionado à escolha da sequência de incrementos (*gaps*), a qual influencia de maneira decisiva o número de comparações e movimentações realizadas ao longo da execução.

Quando se utilizam sequências de incrementos mais simples, como a proposta original de Shell (1959), o pior caso pode atingir ordem quadrática. Ainda assim, ao se adotar estratégias mais elaboradas para a definição dos *gaps*, observa-se uma redução expressiva no custo computacional, tornando o algoritmo mais eficiente em diferentes cenários. Dessa forma, não existe uma fórmula única que descreva seu desempenho, mas apenas estimativas que variam de acordo com a estratégia empregada.

Em virtude dessas características, o Shell Sort mostra-se particularmente vantajoso quando aplicado a conjuntos de dados de grande dimensão. Nesses cenários, sua eficiência tende a superar a de algoritmos de ordenação mais simples, como o Insertion Sort e o Bubble Sort, uma vez que a redução progressiva dos gaps possibilita a organização preliminar dos elementos de forma mais eficaz, acelerando substancialmente as etapas finais de ordenação.

4.5 Merge Sort

No algoritmo *Merge Sort*, dado um vetor A de tamanho N , o processo de ordenação ocorre de forma estruturada e recursiva. A técnica aplica uma lógica de organização progressiva, em que os elementos são gradualmente reorganizados até atingir a sequência completamente ordenada. Essa abordagem garante precisão e estabilidade, apresentando desempenho consistente mesmo em grandes conjuntos de dados. A seguir, será apresentado o algoritmo, seguido da análise de seus casos de complexidade.

	Custo	Vezez
1- if $p < R$	C_1	$O(1)$
2- then $Q \leftarrow \lfloor (P+R)/2 \rfloor$	C_2	$O(1)$
3- MergeSort(A, P, Q)	C_3	$f(\frac{n}{2})$
4- MergeSort(A, P, Q)	C_4	$f(\frac{n}{2})$
5- Merge(A, P, R)	C_5	$O(n)$

Figura 31: Algoritmo Merge Sort

Fonte: Autor

4.5.1 Cálculo Complexidade

Como visto, o custo do algoritmo *Merge Sort* nos casos médio, pior e melhor é representado pela mesma recorrência, dada por $T(n) = 2T(n/2) + n$. Essa expressão indica que, a cada etapa, o vetor é dividido em duas partes de tamanho $n/2$, sendo cada uma ordenada recursivamente, e em seguida ocorre a combinação linear dos resultados.

A seguir, apresenta-se o cálculo detalhado dessa recorrência:

Fórmula Geral $\rightarrow T(n) = 2T\left(\frac{n}{2}\right) + n$
 Provando por recorrência
 $n = 2^m \quad k = \log_2 m \quad T(2^0) = 1$
 Ex: $P(k) = T(2^k) = 2T(2^{k-1}) + 2^k$
 $P(k-1) = T(2^{k-1}) = 2T(2^{k-2}) + 2^{k-1}$
 $\rightarrow P(k) = T(2^k) = 2T(2^{k-1}) + 2^k$
 $CP \rightarrow k$
 $2[2T(2^{k-2}) + 2^{k-1}] + 2^k$
 $2^2 T(2^{k-2}) + 2(2^{k-1})$
 $2^2 T(2^{k-2}) + 2^k$
 $2^k T(2^0) + k(2^k)$
 $2^k + k(2^k)$
 FF $\rightarrow T(n) = n + \log_2(n) \cdot n$

Provando a recorrência por indução
 $P(1) = T(2) = 2^0 + 1 = 2 \quad OK$
 $P(k) = T(2^k) = 2^k + k(2^k)$
 $\rightarrow P(k+1) = T(2^{k+1}) = 2^{k+1} + (k+1)(2^{k+1})$
 $\rightarrow P(k+1) = T(2^{k+1}) = 2T(2^k) + 2^{k+1}$
 $2(2^k + k(2^k)) + 2^{k+1}$
 $2^{k+1} + k(2^{k+1}) + 2^{k+1}$
 $2^{k+1} + (k+1)(2^{k+1})$

Figura 32: Pior, Médio e Melhor caso Merge Sort

Fonte: Autor

Como visto nos cálculos, o algoritmo *Merge Sort* apresenta complexidade de $O(n \log n)$, o que o torna eficiente e viável para grandes volumes de dados. Contudo, seu principal inconveniente está no elevado consumo de memória, uma vez que o processo de combinação exige a criação de estruturas auxiliares para armazenar as subsequências temporárias.

4.5.2 Merge

O *merge* é a etapa responsável por combinar as subsequências já ordenadas, comparando seus elementos e reunindo-os em uma única sequência final em ordem crescente. Esse processo

garante que, ao final da combinação, todos os elementos do vetor estejam corretamente posicionados. A seguir o algoritmo:

Merge	Custo	vezes
1- while ($i < Q+1$ and $j < R+1$)	C1	m
2- if ($v[i] \leq v[j]$)	C2	m-1
3- $w[k++] = v[i++]$	C3	m-1
4- else	C4	m-1
5- $w[k++] = v[j++]$	C5	m-1
6- while ($i < Q$)	C6	m-1
7- $w[k++] = v[i++]$	C7	m-2
8- while ($j < R$)	C8	m-1
9- $w[k++] = v[j++]$	C9	m-2
10- for ($i = P; i < R; i++$)	C10	m-1
11- $v[i] = w[i - P]$	C11	m-2

Figura 33: Algoritmo Merge

Fonte: Autor

Adiante, apresenta-se o cálculo do custo $T(n)$ e a análise dos demais casos, considerando o comportamento do procedimento *merge* em diferentes situações de execução.

$$\begin{aligned}
f(m) &= C_1m + C_2(m-1) + C_3(m-1) + C_4(m-1) + C_5(m-1) + C_6(m-1) + C_7(m-2) \\
&+ C_8(m-1) + C_9(m-2) + C_{10}(m-1) + C_{11}(m-2) \\
&\text{Pior, Médio e Melhor caso} \\
\hookrightarrow f(m) &= (C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 + C_8 + C_9 + C_{10} + C_{11})m + (-C_2 - C_3 - C_4 \\
&- C_5 - C_6 - 2C_7 - C_8 - 2C_9 - C_{10} - 2C_{11}) \\
f(m) &= A'm + B'
\end{aligned}$$

Figura 34: Pior, Médio e Melhor caso Merge

Fonte: Autor

Como visto nos cálculos, o procedimento *merge* apresenta complexidade de $O(n)$ em todos os casos, uma vez que percorre integralmente as subsequências para realizar as comparações e a combinação final dos elementos.

4.5.3 Conclusão da Análise do Merge Sort

Como demonstrado nos cálculos de complexidade do *Merge Sort*, observa-se que se trata de um algoritmo bastante eficiente para grandes volumes de dados, especialmente quando comparado a outros métodos de ordenação baseados em trocas, como o *Bubble Sort* e o *Insertion Sort*. No entanto, é nítido o seu maior custo de memória, uma vez que o processo de combinação requer a utilização de vetores auxiliares durante a execução.

4.6 Quick Sort

Observa-se que N corresponde ao tamanho do vetor. Nota-se que o pseudocódigo do *Quick Sort* é composto por uma verificação e um laço principal que realiza as divisões do vetor em partes menores, o que torna sua estrutura relativamente simples e intuitiva, apesar de sua alta eficiência computacional.

A seguir o pseudo-código com seus custos e vezes correspondentes:

Pseudo-código	Custo	Vezez
1. if $p < R - 1$	C_1	$O(1)$
2. then Q ₄ <i>particiona</i> (A, p, R)	C_2	$O(n)$
3. Quick sort ($A, p, Q - 1$)	C_3	$O(\frac{n}{2})$
4. Quick sort ($A, Q + 1, R$)	C_4	$O(\frac{n}{2})$

Figura 35: Pseudo-Código Quick Sort

Fonte: Autor

É válido ressaltar a complexidade do *particiona* antes de analisarmos os casos do *Quick Sort*. O algoritmo *particiona* possui complexidade $O(n)$, pois, para um vetor de tamanho n , ele precisa percorrer todos os elementos do array uma única vez, posicionando corretamente os elementos em relação ao *pivô* e realizando as trocas necessárias. Por essa razão, sua complexidade é linear. A seguir o seu pseudo-código segundo (ZIVIANI, 2011):

```

Pseudo-código Particiona
1  declare x, w, i, j
2  begin
3    i := Esq ; j := Dir
4    x := A[(i+j) div 2]
5    repeat
6      while x < A[i] do i := i + 1
7      while x > A[j] do j := j - 1
8      if i < j
9        then begin
10           w := A[i] ; A[i] := A[j] ; A[j] := w
11           i := i + 1 ; j := j - 1
12        end

```

Figura 36: Pseudo-Código Particiona

Fonte: Ziviani

4.6.1 Fórmula Geral

A fórmula geral do *Quick Sort* pode ser expressa pela recorrência $2T\left(\frac{n}{2}\right) + n$ no melhor caso e $T(n-1) + n$ no pior caso, representando, respectivamente, os custos das divisões equilibradas e totalmente desequilibradas durante o processo de ordenação.

4.6.2 Melhor Caso

O melhor caso ocorre quando as partições possuem sempre o mesmo tamanho, conhecido como partições balanceadas, sendo representado pela recorrência mencionada anteriormente. A seguir, apresenta-se o cálculo do melhor caso:

Melhor Caso

$$T(m) = 2T\left(\frac{m}{2}\right) + m$$

$$m = 2^k \quad k = \log_2 m \quad T(2^0) = 1$$

$$\text{Exp} \rightarrow T(2^k) = 2T(2^{k-1}) + 2^k$$

$$T(2^{k-1}) = 2T(2^{k-2}) + 2^{k-1}$$

$$T(2^k) = 2T(2^{k-1}) + 2^k$$

$$2[2T(2^{k-2}) + 2^{k-1}] + 2^k$$

$$2^2 T(2^{k-2}) + 2(2^k)$$

$$2^i T(2^{k-i}) + i(2^k)$$

$$\text{CP} \rightarrow i = k$$

$$2^k T(2^0) + k(2^k)$$

$$2^k + k(2^k)$$

$$\text{FF} \rightarrow m + \log_2(m)$$

Ind

$$P(1) = T(2^0) = 1 + 0 + 0k$$

$$P(k) = T(2^k) = 2^k + k(2^k)$$

$$P(k+1) = T(2^{k+1}) = 2^{k+1} + (k+1)(2^{k+1})$$

$$\rightarrow T(2^{k+1}) = 2T(2^k) + 2^{k+1}$$

$$2[2^k + k(2^k)] + 2^{k+1}$$

$$2^{k+1} + (k(2^{k+1})) + 2^{k+1}$$

$$2^{k+1} + (k+1)(2^{k+1})$$

Figura 37: Melhor Caso Quick Sort

Fonte: Autor

Nos cálculos acima, nota-se que a recorrência resulta na fórmula fechada $n + n \log n$, o que implica que, no melhor caso, o *Quick Sort* apresenta uma análise assintótica de $O(n \log n)$.

4.6.3 Médio Caso

O caso médio do *Quick Sort* é mais complexo de ser representado por uma recorrência exata. Contudo, segundo Sedgewick e Flajolet (1996), o número médio de comparações pode ser aproximado por $1,386 n \log n - 0,846 n$, o que mantém, de forma assintótica, o tempo de execução na ordem de $O(n \log n)$.

4.6.4 Pior Caso

O pior caso do *Quick Sort* ocorre quando o elemento *pivô* escolhido é sempre o menor ou o maior elemento do subarranjo, tornando o processo completamente diferente do melhor caso, no qual as partições são balanceadas aqui, elas se tornam totalmente desbalanceadas. Essa situação acontece quando o vetor já se encontra ordenado de forma crescente ou decrescente, resultando na recorrência $T(n-1) + n$. A seguir, apresentam-se os cálculos referentes ao pior caso:

Pior Caso

$$f(m) = f(m-1) + m \quad f(1) = 1$$

$$f(m-1) = f(m-2) + m-1$$

$$\leadsto f(k) = f(k-1) + f(k-2) + (k-1) + k$$

$$f(k-1) = f(k-2) + (k-1) + k$$

$$f(k-2) = f(k-3) + (k-2) + k$$

$$\vdots$$

$$f(1) = 1 + 1 = 2$$

$$CP \leadsto \sum_{i=1}^{k-1} f(i) + \dots + k$$

$$PA \leadsto S_m = \frac{(1+k)k}{2} \leadsto \frac{k^2+k}{2}$$

$$FF \leadsto \frac{m^2+m}{2}$$

$$Ind \leadsto P(1) = f(1) = \frac{1+1}{2} = 1 OK$$

$$P(k) = f(k) = \frac{k^2+k}{2}$$

$$P(k+1) = f(k+1) = \frac{(k+1)(k+2)}{2}$$

$$f(k+1) = f(k) + k$$

$$\frac{k^2+k}{2} + (k+1) \leadsto \frac{k^2+3k+2}{2} \leadsto \frac{(k+1)(k+2)}{2}$$

Figura 38: Pior Caso Quick Sort

Fonte: Autor

Nota-se, nos cálculos acima, que a fórmula fechada do pior caso é dada por $\frac{n^2+n}{2}$, o que resulta em uma análise assintótica de $O(n^2)$, representando um tempo de execução consideravelmente mais lento em comparação aos demais casos.

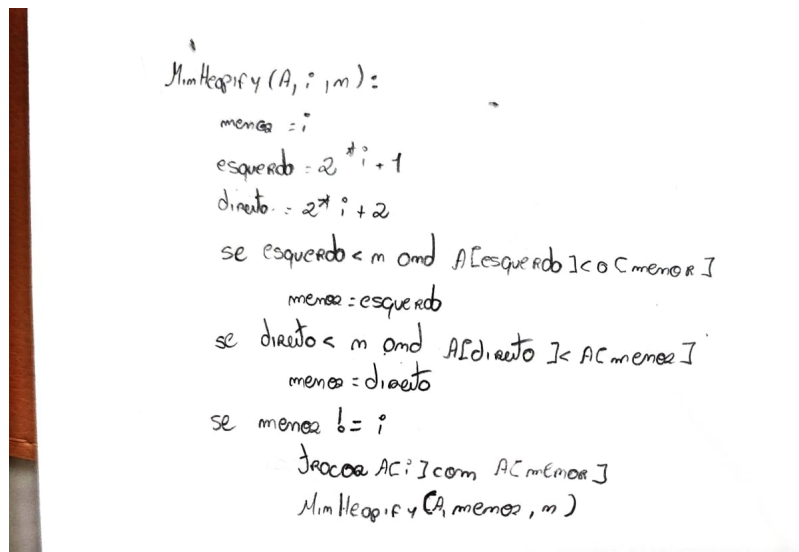
4.6.5 Conclusão da Análise do Quick Sort

Como mostrado anteriormente, o *Quick Sort* demonstra alta eficiência em entradas aleatórias, ou seja, em seu caso médio sendo que, evidentemente, no melhor caso o desempenho também é excelente. Contudo, fora de um estudo controlado, as situações reais tendem a se comportar de forma semelhante ao caso médio, o que torna o *Quick Sort* um algoritmo extremamente

eficiente e amplamente utilizado na prática. Ainda assim, é importante destacar o tempo de execução consideravelmente maior em seu pior caso.

4.7 Heap Sort

O algoritmo *Heap Sort*, de forma geral, depende de funções auxiliares que garantem a manutenção da propriedade de *heap*, seja ele do tipo máximo ou mínimo. A função *build_heap* é responsável por construir a estrutura inicial, reorganizando o vetor de modo que cada nó obedeça à hierarquia definida no *max-heap*, o pai possui valor maior que seus filhos, enquanto no *min-heap*, o pai possui valor menor. Já a função *heapify* atua de forma iterativa, restaurando essa propriedade sempre que ocorrem trocas entre elementos durante o processo de ordenação. Em conjunto, essas operações permitem que os elementos sejam gradualmente posicionados em suas devidas posições, resultando na ordenação completa do vetor. A seguir, são apresentados os algoritmos e os cálculos de complexidade das principais funções utilizadas na implementação do algoritmo, baseado no heap mínimo, começando por MinHeapify.



```
MinHeapify(A, i, m):  
    menor = i  
    esquerdo = 2 * i + 1  
    direito = 2 * i + 2  
    se esquerdo < m and A[esquerdo] < A[menor]  
        menor = esquerdo  
    se direito < m and A[direito] < A[menor]  
        menor = direito  
    se menor != i  
        troca A[i] com A[menor]  
        MinHeapify(A, menor, m)
```

Figura 39: Algoritmo MinHeapify

Fonte: Autor

Cálculo tempo MinHeapify

$$T(m) = T\left(\frac{m}{2}\right) + 1 \quad m = \left(\frac{3}{2}\right)^k \quad T\left(\frac{3^0}{2}\right) = 1$$

$$T(m) = T\left(\frac{m}{2}\right) + 1 \quad k = \log_{\frac{3}{2}} m$$

$$\begin{aligned} \text{Exp} \rightarrow T\left(\frac{3^k}{2}\right) &= T\left(\frac{3^{k-1}}{2}\right) + 1 \\ T\left(\frac{3^{k-1}}{2}\right) &= T\left(\frac{3^{k-2}}{2}\right) + 1 \\ T\left(\frac{3^{k-2}}{2}\right) &= T\left(\frac{3^{k-3}}{2}\right) + 1 \end{aligned} \quad \begin{aligned} T\left(\frac{3^k}{2}\right) &= T\left(\frac{3^{k-1}}{2}\right) + 1 \\ &\approx T\left(\frac{3^{k-2}}{2}\right) + 2(1) \\ &= T\left(\frac{3^{k-1}}{2}\right) + 1(1) \\ T\left(\frac{3^0}{2}\right) + k &\approx 1 + k \\ \text{ff} \rightarrow T(m) &= \log_{\frac{3}{2}} m + 1 \end{aligned} \quad CP \rightarrow 1 = k$$

$$\text{Ind } P(1) = T\left(\frac{3^0}{2}\right) = \log_{\frac{3}{2}} \left(\frac{3^0}{2}\right) + 1 = 1 \quad OK$$

$$\begin{aligned} P(k) &= T\left(\frac{3^k}{2}\right) = k + 1 \\ P(k+1) &= T\left(\frac{3^{k+1}}{2}\right) = k + 2 \\ T\left(\frac{3^{k+1}}{2}\right) &= T\left(\frac{3^k}{2}\right) + 1 \\ &= k + 1 + 1 \\ &= k + 2 \end{aligned}$$

Figura 40: Cálculo Complexidade MinHeapify

Fonte: Autor

Conforme observado no algoritmo apresentado, cada subárvore gerada possui, em média, tamanho aproximado de $\frac{2n}{3}$. Considerando essa proporção e a constante associada às operações de manutenção do *heap*, obtém-se a recorrência descrita anteriormente, que representa o custo total do algoritmo.

Adiante o BuildMinHeap:

```

BuildMinHeap(A, n)
  int n = tamanho_vet
  para (int i = n/2 até 0)
    MinHeapify(A, i, n)

```

Figura 41: Algoritmo BuildMinHeap

Fonte: Autor

Conforme observado no algoritmo apresentado, a estrutura contém apenas um laço *for*, o que indica uma complexidade temporal de ordem $O(n)$, uma vez que cada elemento do vetor é processado exatamente uma vez.

Adiante, será apresentado o algoritmo Heapsort mínimo.

```

HeapSortMin(A, n)
  BuildMinHeap(A, n)
  para (i = n-1 até 1)
    troca A[0] com A[i]
    MinHeapify(A, 0, i)

```

Figura 42: Algoritmo HeapSortMin

Fonte: Autor

Como observado nos algoritmos apresentados, o *Heap Sort* realiza inicialmente a chamada da função *Build-Min-Heap*, cuja complexidade é $O(n)$. Em seguida, o laço principal executa n chamadas da função *Min-Heapify*, cada uma com complexidade $O(\log n)$. Dessa forma, a complexidade total do *Heap Sort Mínimo* é de $O(n \log n)$, o que o torna um algoritmo eficiente e estável em termos de desempenho assintótico, uma vez que seus melhores, médios e piores casos apresentam a mesma ordem de complexidade.

4.7.1 Conclusão da Análise do Heap Sort

Com base na análise realizada e na literatura especializada, observa-se que o *Heap Sort* apresenta excelente desempenho assintótico, com tempo médio e pior caso de $O(n \log n)$ (CORMEN, 2012). Sua estrutura baseada em *heap* garante ordenação estável e previsível, tornando-o uma opção eficiente e confiável para grandes volumes de dados.

5 TABELA E GRÁFICO

Nesta seção, são apresentados os resultados obtidos por meio de tabelas e gráficos referentes aos algoritmos analisados. As entradas utilizadas possuem tamanhos de 10, 100, 1.000, 10.000, 100.000 e 1.000.000 elementos, armazenados em vetores e gerados em três cenários distintos: crescente, decrescente e aleatório. Os experimentos foram executados em um ambiente computacional com as seguintes especificações: processador *Intel Core i5*, placa gráfica *NVIDIA RTX 3050* e memória principal de *16 GB de RAM*.

5.1 Insertion Sort

A seguir, apresentam-se a tabela e o gráfico que ilustram o desempenho do algoritmo Insertion Sort.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0	0	0	0.006
Decrescente	0	0	0.003	0.17	17.069	1907.9
Aleatória	0	0	0.001	0.084	33.671	897.195

Tabela 2: Tabela de tempo por segundo do algoritmo Insertion Sort

Fonte: Autor

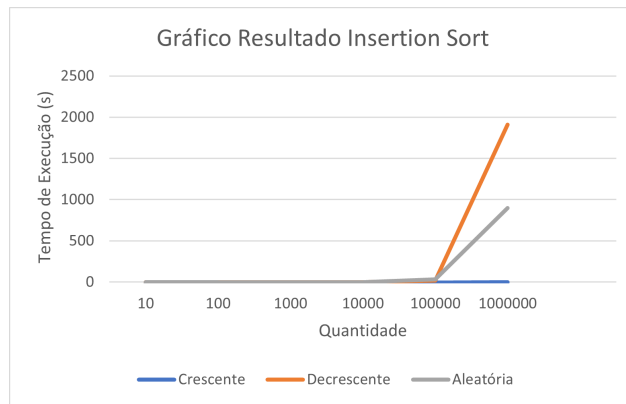


Figura 43: Gráfico de resultados do Insertion Sort

Fonte: Autor

Conforme apresentado na Tabela 2 e na Figura 43, observa-se que o algoritmo Insertion Sort, nos cenários de médio caso (aleatórias) e pior caso (decrescentes), apresenta desempenho significativamente lento. Dessa forma, confirma-se a inviabilidade de sua aplicação em conjuntos de dados desordenados, em consonância com a discussão da Seção 4.1.

5.2 Selection Sort

Na sequência, são exibidos a tabela e o gráfico referentes ao desempenho do algoritmo Selection Sort.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0.04	0.18	17.788	1813.79
Decrescente	0	0	0.002	0.176	17.866	1798.06
Aleatória	0	0	0.003	0.18	18.506	1803.79

Tabela 3: Tabela de tempo por segundo do algoritmo Selection Sort

Fonte: Autor

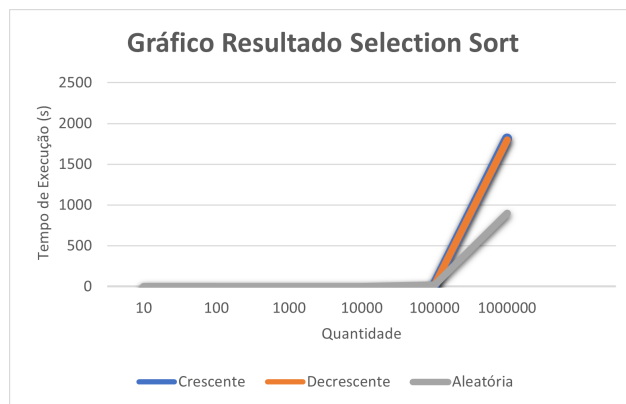


Figura 44: Gráfico de resultados Selection Sort

Fonte: Autor

Como mostrado na Tabela 3 e na Figura 44, observa-se que o algoritmo Selection Sort apresenta tempos de execução bastante semelhantes. Esse resultado evidencia que, para quanto maior o volume de dados, o Selection Sort se torna um algoritmo ainda mais lento, corroborando o que foi discutido na Seção 4.2.

5.3 Bubble Sort

A seguir, apresentam-se a tabela e o gráfico que demonstram o desempenho do algoritmo Bubble Sort.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0.003	0.243	24.168	2367.3
Decrescente	0	0	0.006	0.422	42.125	4232.65
Aleatória	0	0	0.05	0.403	44.615	4250.44

Tabela 4: Tabela de tempo por segundo do algoritmo Bubble Sort

Fonte: Autor



Figura 45: Gráfico de resultado Bubble Sort

Fonte: Autor

Como mostrado na Tabela 4 e na Figura 45, verifica-se que o Bubble Sort apresenta desempenho insatisfatório em grandes volumes de dados. Nessas condições, o tempo de execução torna-se excessivamente elevado, o que evidencia a inviabilidade prática do algoritmo, conforme discutido na Seção 4.3.

5.4 Shell Sort

Na sequência, são exibidos a tabela e o gráfico referentes ao desempenho obtido pelo algoritmo Shell Sort.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0	0	0.01	0.09
Decrescente	0	0	0	0.001	0.011	0.129
Aleatória	0	0	0.001	0.002	0.029	0.397

Tabela 5: Tabela de tempo por segundo do algoritmo Shell Sort

Fonte: Autor



Figura 46: Gráfico de resultado Shell Sort

Fonte: Autor

Como mostrado na Tabela 5 e na Figura 46, verifica-se que o Shell Sort apresenta elevado desempenho mesmo com o aumento da quantidade de dados, configurando-se como uma opção eficiente para volumes médios. Em comparação a algoritmos mais simples, como o Insertion Sort, destaca-se por lidar de forma mais eficaz com grandes conjuntos, conforme evidenciado na Seção 4.4.

5.5 Comparação dos Algoritmos Baseado em Trocas

A seguir, será apresentada a análise de todos os algoritmos considerando uma determinada entrada.

5.5.1 Crescente

	10	100	1000	10000	100000	1000000
Insertion	0	0	0	0	0	0,006
Selection	0	0	0.04	0.18	17.788	1813.79
Bubble	0	0	0.003	0.243	24.168	2367.3
Shell	0	0	0	0	0.01	0.09

Tabela 6: Tabela de tempo por segundo de todos os algoritmos em ordem crescente

Fonte: Autor

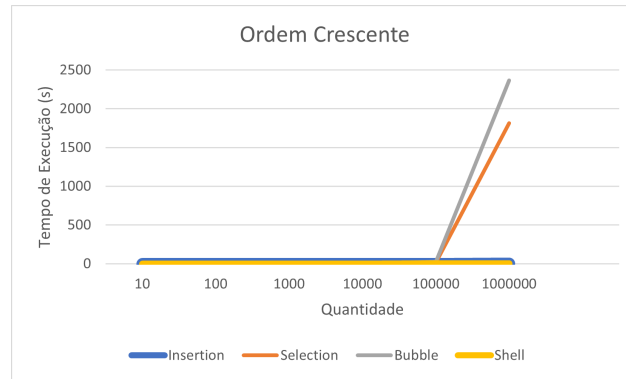


Figura 47: Gráfico de resultados de todos os algoritmos em ordem crescente

Fonte: Autor

A partir da Figura 48, nota-se que os algoritmos Insertion Sort e Shell Sort apresentaram tempos de execução bastante reduzidos mesmo com o aumento do número de elementos, demonstrando maior eficiência na ordenação em ordem crescente. Em contraste, os algoritmos Selection Sort e Bubble Sort tiveram crescimento expressivo no tempo de execução para volumes elevados, evidenciando sua limitação diante de grandes conjuntos de dados. Esses resultados reforçam a superioridade de métodos mais otimizados, como o Shell Sort, em cenários que demandam maior escalabilidade.

5.5.2 Decrescente

	10	100	1000	10000	100000	1000000
Insertion	0	0	0.003	0.17	17.069	1907.9
Selection	0	0	0.002	0.176	17.866	1798.06
Bubble	0	0	0.006	0.422	42.125	4232.65
Shell	0	0	0	0.001	0.011	0.129

Tabela 7: Tabela por segundo de todos os algoritmos em ordem decrescente

Fonte: Autor

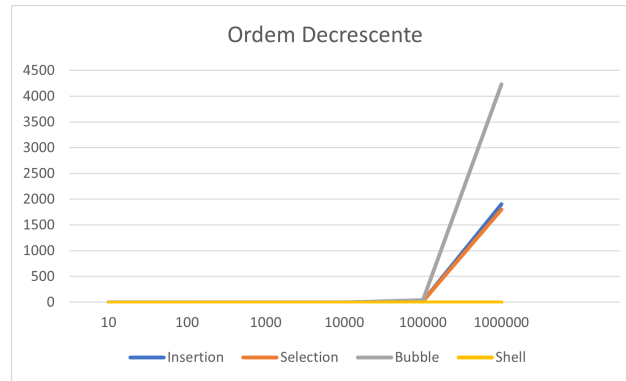


Figura 48: Gráfico de resultados de todos os algoritmos em ordem decrescente

Fonte: Autor

Na ordenação em ordem decrescente, observa-se um comportamento semelhante ao do caso crescente, porém com diferenças mais acentuadas entre os algoritmos. O Shell Sort manteve desempenho consistente, apresentando tempos de execução muito inferiores em relação aos demais, mesmo para grandes volumes de dados. Já o Bubble Sort mostrou-se novamente o menos eficiente, com crescimento expressivo no tempo de execução à medida que a quantidade de elementos aumentou. O Insertion Sort e o Selection Sort apresentaram desempenho intermediário, mas ainda significativamente inferior ao Shell Sort, evidenciando sua limitação diante de entradas em ordem inversa.

5.5.3 Aleatória

	10	100	1000	10000	100000	1000000
Insertion	0	0	0.001	0.084	33.671	897.195
Selection	0	0	0.003	0.18	18.506	1803.79
Bubble	0	0	0.05	0.403	44.615	4250.44
Shell	0	0	0.001	0.002	0.029	0.397

Tabela 8: Tabela por segundo de todos os algoritmos em ordem aleatória

Fonte: Autor

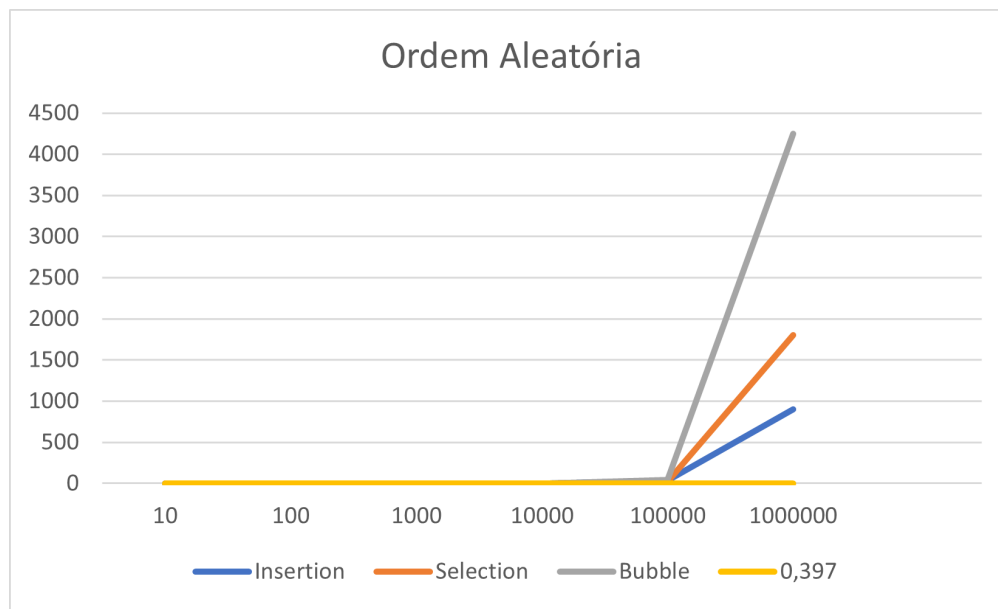


Figura 49: Gráfico de resultados de todos os algoritmos em ordem aleatória

Fonte: Autor

Na ordenação de dados dispostos de forma aleatória, o Shell Sort novamente se destacou por apresentar tempos de execução significativamente menores em comparação aos demais algoritmos, mesmo para volumes elevados. O Bubble Sort manteve-se como o algoritmo de pior desempenho, com crescimento acentuado no tempo de processamento conforme aumentou a quantidade de elementos. Já o Insertion Sort e o Selection Sort exibiram desempenhos intermediários, porém ainda muito inferiores ao Shell Sort, evidenciando suas limitações frente a entradas sem padrão definido.

5.6 Merge Sort

A seguir, são apresentadas a tabela e o gráfico com os resultados obtidos para o algoritmo *Merge Sort*.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0	0.004	0.035	0.376
Decrescente	0	0	0.001	0.004	0.036	0.379
Aleatória	0	0	0	0.005	0.045	0.525

Tabela 9: Tabela por segundo do algoritmo Merge Sort

Fonte: Autor

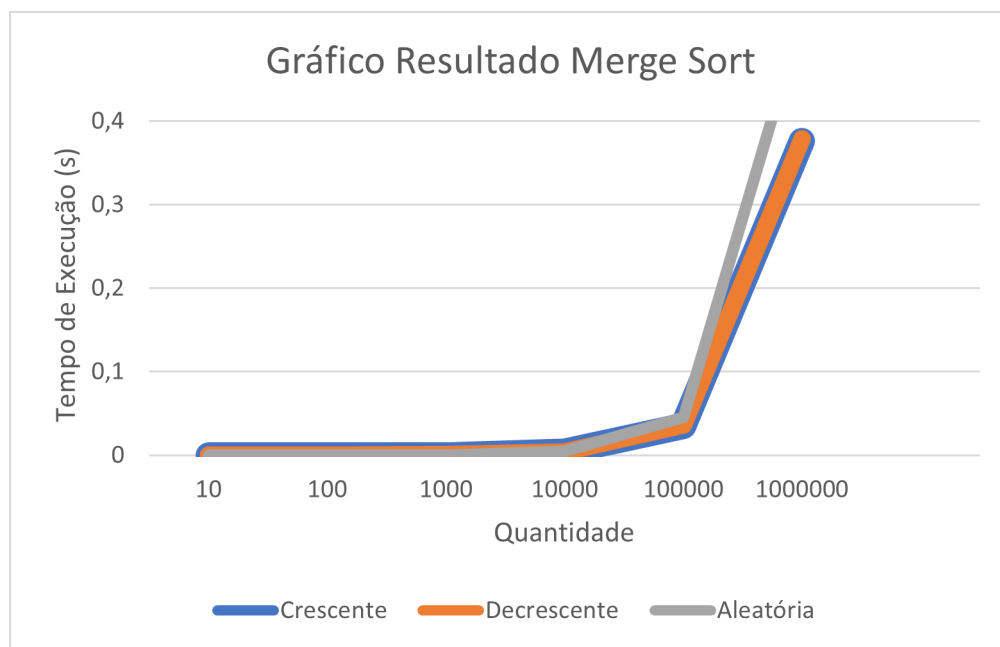


Figura 50: Gráfico de resultado do algoritmo Merge Sort

Fonte: Autor

Como apresentado na Tabela 9 e na Figura 50, observa-se que o *Merge Sort* demonstra excelente desempenho em grandes volumes de dados, evidenciando sua eficiência em termos de tempo de execução. Entretanto, conforme discutido na Seção 4.5, o algoritmo demanda considerável quantidade de memória devido ao uso de vetores auxiliares, o que pode torná-lo menos viável em contextos onde o consumo de memória seja uma limitação significativa.

5.7 Quick Sort

A seguir, apresentam-se a tabela e o gráfico referentes às três versões do *Quick Sort*, permitindo a comparação de seus desempenhos em diferentes cenários de execução.

5.7.1 Quick Sort Versão 1

A seguir, apresenta-se a tabela e o gráfico do Quick Sort com o *particiona*, considerando o *pivô* como o primeiro elemento do vetor.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0.001	0.065	6.73	698.915
Decrescente	0	0	0.001	0.065	6.717	682.403
Aleatória	0	0	0	0.002	0.012	0.146

Tabela 10: Tabela Quick Sort Versão 1

Fonte: Autor

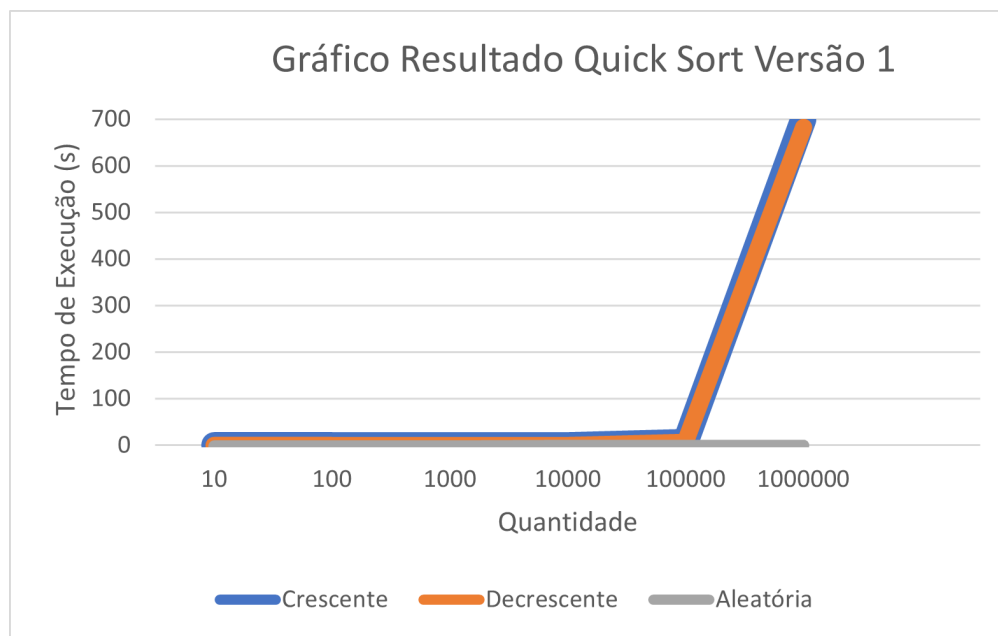


Figura 51: Gráfico Quick Sort Versão 1

Fonte: Autor

Como apresentado na Tabela 10 e na Figura 51, nota-se que o algoritmo que escolhe o *pivô* como o primeiro elemento torna-se inviável para grandes volumes de dados, especialmente quando se busca maior velocidade de processamento.

5.7.2 Quick Sort Versão 2

A seguir, apresentam-se a tabela e o gráfico referentes ao *Quick Sort* utilizando a função *particiona*, na qual o *pivô* é definido como a média entre o primeiro e o último elemento do vetor, buscando uma divisão mais equilibrada entre as partições.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0.	0	0.003	0.033
Decrescente	0	0	0.	0	0.003	0.036
Aleatória	0	0	0.001	0.001	0.011	0.128

Tabela 11: Tabela Quick Sort Versão 2

Fonte: Autor

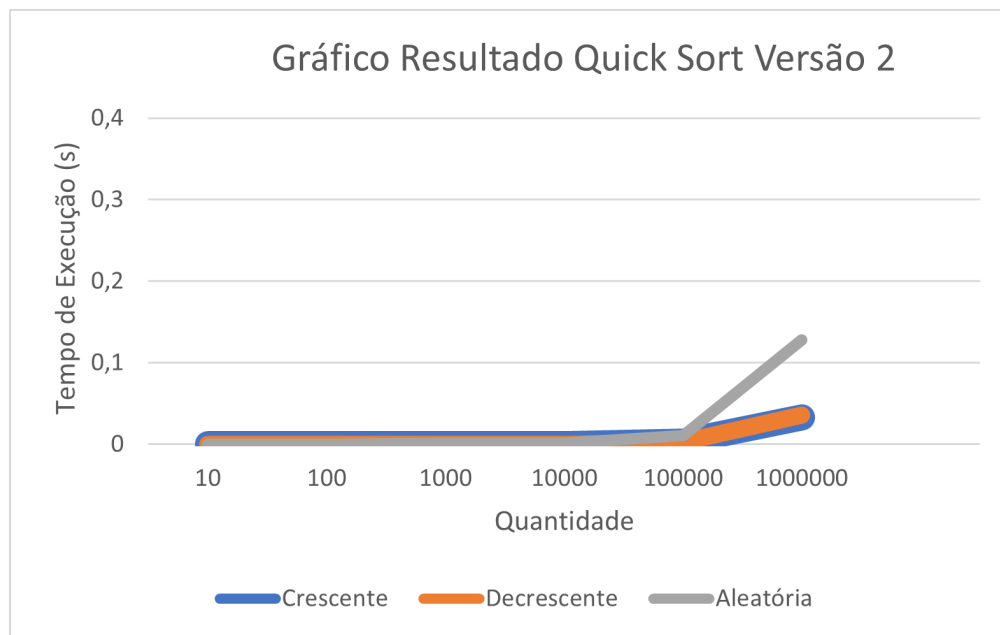


Figura 52: Gráfico Quick Sort Versão 2

Fonte: Autor

Como apresentado na Tabela 11 e na Figura 52, nota-se que, para as diferentes entradas, não há variações significativas em relação ao tempo de execução, mantendo-se um desempenho satisfatório em todos os casos analisados.

5.7.3 Quick Sort Versão 3

A seguir, apresentam-se a tabela e o gráfico referentes ao *Quick Sort* implementado com a função *particiona*, na qual o *pivô* é escolhido de forma randômica.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0.	0	0.005	0.102
Decrescente	0	0	0.	0.001	0.006	0.123
Aleatória	0	0	0	0.001	0.013	0.148

Tabela 12: Tabela Quick Sort Versão 3

Fonte: Autor

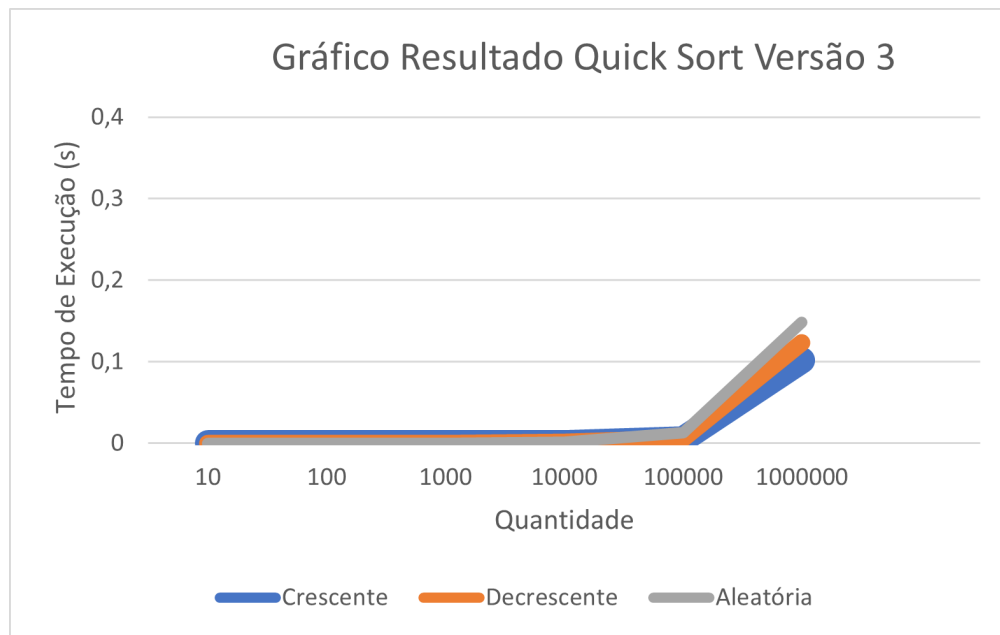


Figura 53: Gráfico Quick Sort Versão 3

Fonte: Autor

Como apresentado na Tabela 12 e na Figura 53, observa-se que o desempenho desta versão do *Quick Sort* assemelha-se bastante ao da Versão 2, mantendo tempos de execução estáveis e satisfatórios, sem grandes variações entre as diferentes entradas.

5.7.4 Análise das Três Versões em Conjunto

Como mencionado anteriormente, este estudo apresenta três versões do *Quick Sort*, que se diferenciam pela forma de escolha do *pivô*: na primeira, é selecionado o primeiro elemento do vetor; na segunda, a média entre os extremos; e, na terceira, um elemento escolhido de forma aleatória. A seguir as tabelas e gráficos comparando as três versões nas diferentes entradas.

Segue adiante em Ordem Crescente:

	10	100	1000	10000	100000	1000000
Quick 1	0	0	0.001	0.065	6.73	698.915
Quick 2	0	0	0	0	0.003	0.033
Quick 3	0	0	0	0	0.005	0.102

Tabela 13: Tabela de Comparação das Versões do Quick em Ordem Crescente

Fonte: Autor

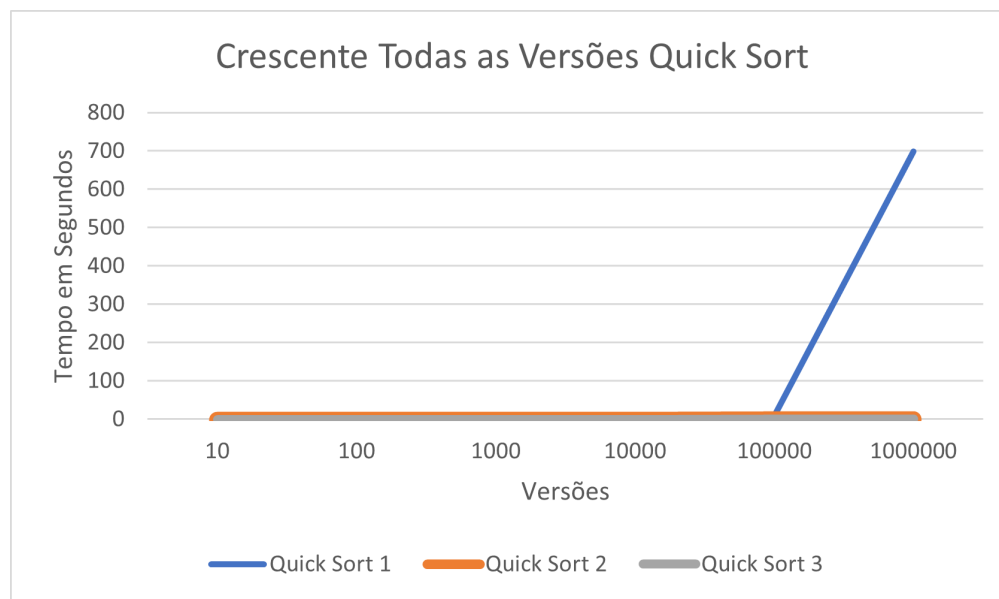


Figura 54: Gráfico de Comparação das Versões do Quick Sort em Ordem Crescente

Fonte: Autor

Adiante em Ordem Decrescente:

	10	100	1000	10000	100000	1000000
Quick 1	0	0	0.001	0.065	6.717	682.403
Quick 2	0	0	0	0	0.003	0.036
Quick 3	0	0	0	0	0.013	0.148

Tabela 14: Tabela de Comparação das Versões do Quick em Ordem Decrescente

Fonte: Autor

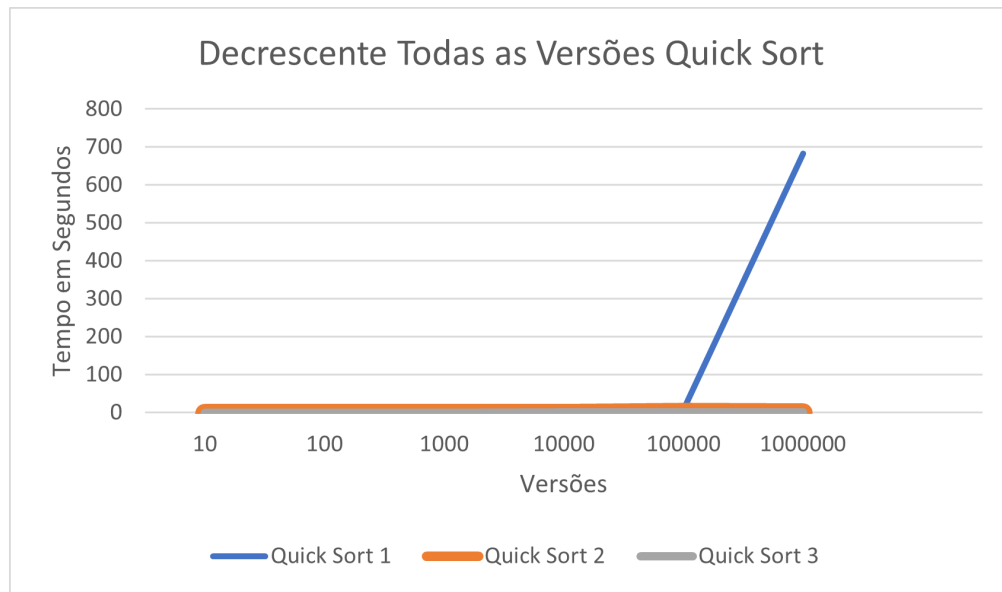


Figura 55: Gráfico de Comparação das Versões do Quick Sort em Ordem Derescente

Fonte: Autor

A seguir em Ordem Aleatória:

	10	100	1000	10000	100000	1000000
Quick 1	0	0	0	0.002	0.012	0.146
Quick 2	0	0	0.001	0.001	0.011	0.128
Quick 3	0	0	0	0.001	0.013	0.0148

Tabela 15: Tabela de Comparação das Versões do Quick em Ordem Aleatória

Fonte: Autor

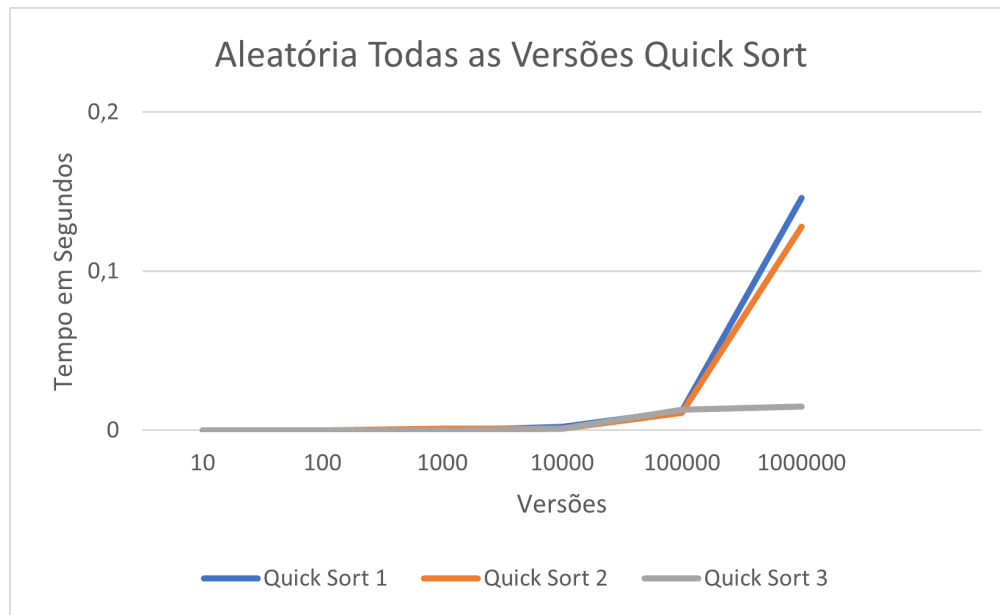


Figura 56: Gráfico de Comparação das Versões do Quick Sort em Ordem Aleatória

Fonte: Autor

Nota-se que a Versão 1 do *Quick Sort* apresenta desempenho inferior em todos os tipos de entrada. Em seguida, a Versão 2 mostra uma melhora significativa, enquanto a Versão 3 mantém tempos de execução bastante semelhantes entre as diferentes entradas. Vale ressaltar que todas as versões lidaram melhor com entradas aleatórias, pois não se observaram variações expressivas nos tempos de execução, confirmando o que foi discutido na Seção 4.5 e Seção 4.6.5. Assim, considerando sua complexidade de $O(n \log n)$ para entradas aleatórias, comprova-se que o *Quick Sort* é, de fato, um excelente algoritmo de ordenação em contextos reais.

5.8 Comparação dos Algoritmos Baseados em Divisão e Conquista

A seguir, terá a apresentação da análise dos algoritmos baseado em divisão e conquista.

5.8.1 Crescente

	10	100	1000	10000	100000	1000000
Quick 1	0	0	0.001	0.065	6.73	698.915
Quick 2	0	0	0	0	0.003	0.033
Quick 3	0	0	0	0	0.005	0.102
Merge	0	0	0	0.004	0.035	0.376

Tabela 16: Tabela de Tempo Por Segundo de Todos os Algoritmos em Ordem Crescente

Fonte: Autor

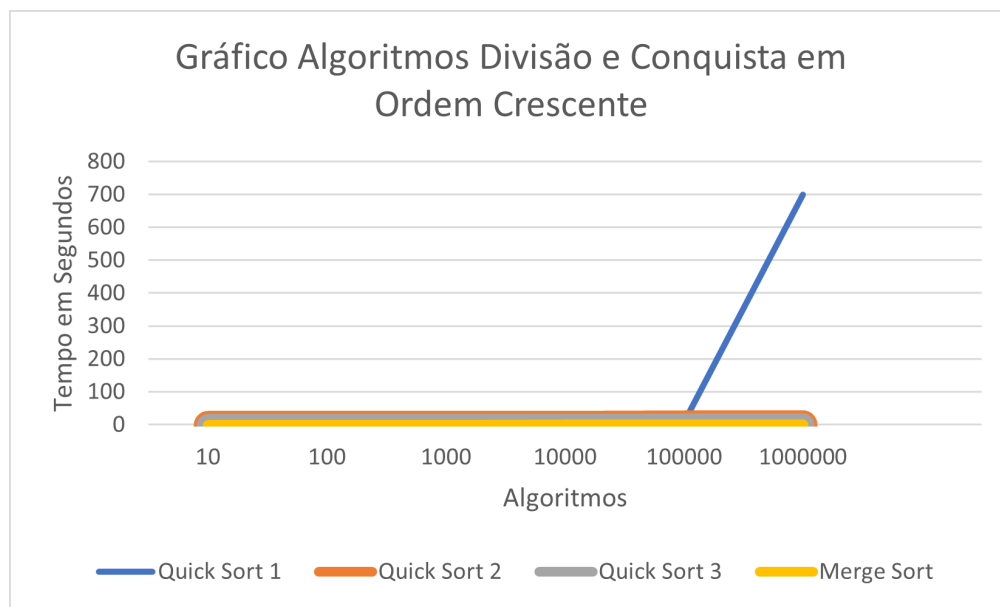


Figura 57: Gráfico de Resultado de Todos os Algoritmos em Ordem Crescente

Fonte: Autor

Observa-se que, para entradas em ordem crescente, a versão 2 do *Quick Sort* cujo pivô é definido pela média dos elementos extremos apresenta melhor desempenho, demonstrando menor variação nos tempos de execução. Por outro lado, a versão 1, que utiliza o primeiro elemento como pivô, não é recomendada nesse tipo de entrada, pois tende a gerar partições desequilibradas e, conseqüentemente, maior tempo de processamento.

5.8.2 Decrescente

	10	100	1000	10000	100000	1000000
Quick 1	0	0	0.001	0.065	6.717	682.403
Quick 2	0	0	0	0	0.003	0.036
Quick 3	0	0	0	0.001	0.013	0.148
Merge	0	0	0,001	0.004	0.036	0.379

Tabela 17: Tabela de Tempo Por Segundo de Todos os Algoritmos em Ordem Decrescente

Fonte: Autor

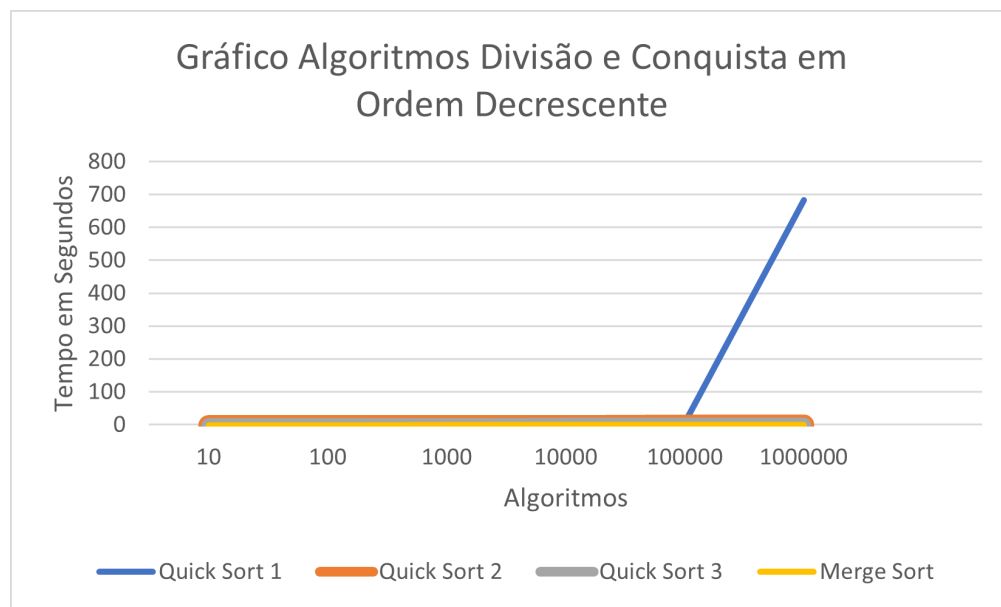


Figura 58: Gráfico de Resultado de Todos os Algoritmos em Ordem Decrescente

Fonte: Autor

No caso das entradas em ordem decrescente, o comportamento é semelhante ao observado para as entradas crescentes: a versão 1 do *Quick Sort* permanece como a menos eficiente, enquanto as demais versões mantêm tempos de execução estáveis, sem grandes variações de desempenho.

5.8.3 Aleatória

	10	100	1000	10000	100000	1000000
Quick 1	0	0	0	0.002	0.012	0.146
Quick 2	0	0	0.001	0.001	0.011	0.128
Quick 3	0	0	0	0.001	0.013	0.0148
Merge	0	0	0	0.005	0.045	0.525

Tabela 18: Tabela de Tempo Por Segundo de Todos os Algoritmos em Ordem Aleatória

Fonte: Autor

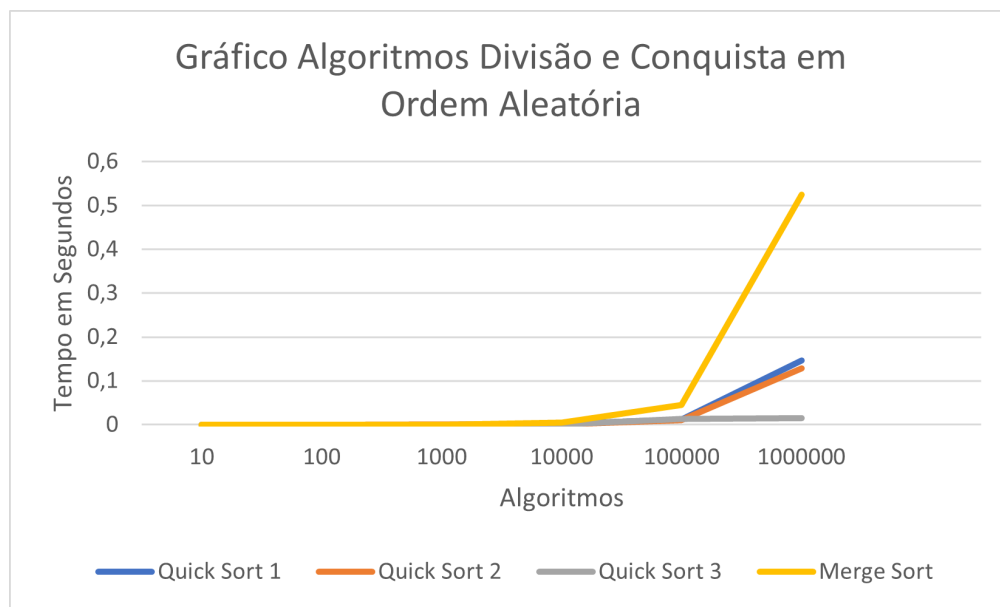


Figura 59: Gráfico de Resultado de Todos os Algoritmos em Ordem Aleatória

Fonte: Autor

Verifica-se que, para entradas aleatórias, todos os algoritmos apresentam melhor desempenho em relação aos casos ordenados. Contudo, mesmo não havendo um aumento expressivo no tempo de execução, o *Merge Sort* mostrou-se menos eficiente em comparação aos demais. A versão 3 do *Quick Sort* manteve desempenho estável, sem grandes variações nos tempos observados.

5.9 Heap Sort

Na sequência, são apresentadas a tabela e o gráfico que representam o desempenho do algoritmo *Heap Sort*.

	10	100	1000	10000	100000	1000000
Crescente	0	0	0.001	0.002	0.02	0.248
Decrescente	0	0	0.	0.002	0.02	0.253
Aleatória	0	0.001	0	0.002	0.024	0.324

Tabela 19: Tabela Heap Sort

Fonte: Autor

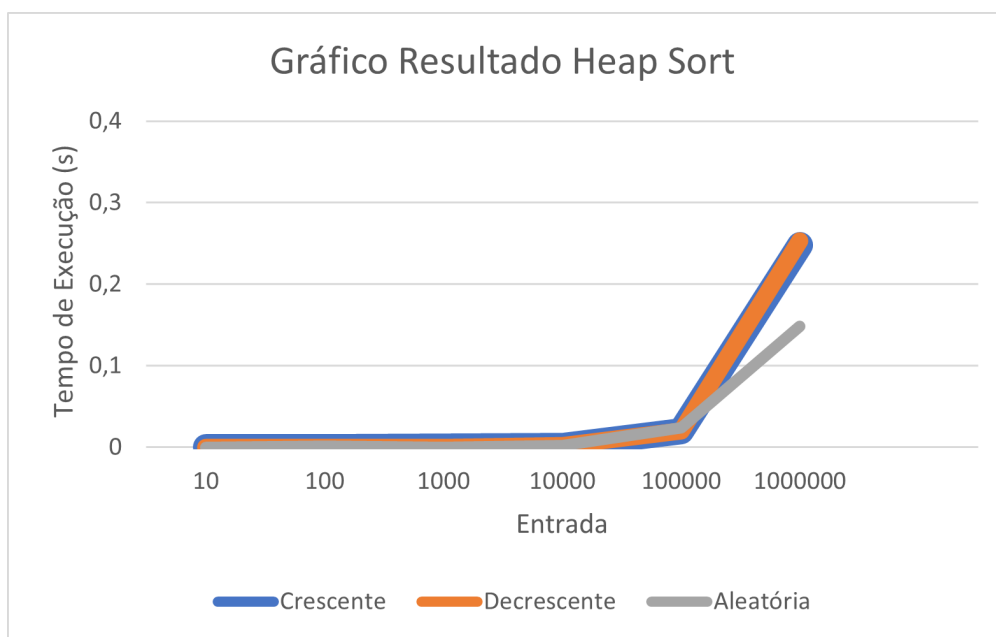


Figura 60: Gráfico de Resultado do Heap Sort

Fonte: Autor

Conforme apresentado na Tabela 19 e na Figura 60, observa-se que o algoritmo *Heap Sort* apresenta desempenho consistente em todos os casos analisados, com tempos de execução bastante próximos entre as diferentes configurações de entrada. Dessa forma, evidencia-se que o *Heap Sort* cumpre de maneira eficiente o propósito a que se propõe, demonstrando estabilidade e bom comportamento independente da ordenação inicial dos dados.

6 CONCLUSÃO

O objetivo deste projeto foi realizar uma análise comparativa do desempenho dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort*, *Shell Sort*, *Merge Sort*, *Quick Sort* e *Heap Sort*, a fim de identificar em quais situações o uso de cada um se mostra mais viável. O relatório apresenta uma breve explicação teórica de cada algoritmo, o cálculo de suas complexidades, a análise dos tempos de execução obtidos para diferentes tipos de entrada e, por fim, uma conclusão baseada nos resultados observados.

De acordo com as análises realizadas, nas entradas em ordem crescente os algoritmos *Insertion Sort* e *Shell Sort* demonstraram melhor desempenho, evidenciando sua eficiência em conjuntos de dados previamente ordenados. Nesses casos, tais métodos se destacam entre os algoritmos baseados em trocas, enquanto outros como o *Bubble Sort* mantêm tempos de execução elevados independentemente da entrada. Já os algoritmos baseados em divisão e conquista mostraram-se mais eficientes em termos de tempo, embora apresentem implementação mais complexa. Essa complexidade, no entanto, é compensada por seu ótimo desempenho, raramente ultrapassando a ordem de $O(n \log n)$. Observa-se também que o *Merge Sort* e o *Heap Sort* mantêm bom desempenho em entradas crescentes, enquanto o *Quick Sort* pode sofrer degradação quando ocorre uma má escolha de pivô. Nas entradas decrescentes, os algoritmos de troca tornam-se significativamente menos eficientes, ao passo que os de divisão e conquista lidam melhor com esse tipo de situação. Em entradas aleatórias, o *Shell Sort* se sobressai entre os algoritmos de troca, mas é nesse contexto que os algoritmos de divisão e conquista evidenciam seu potencial. O *Quick Sort*, especialmente em sua versão que utiliza o pivô como média dos extremos, apresentou o melhor desempenho geral, destacando-se como a opção mais eficiente para dados de natureza variada.

Constata-se, portanto, a importância de compreender o comportamento e as características de cada algoritmo, de modo a aplicá-los adequadamente conforme o tipo e o tamanho do conjunto de dados. Essa escolha consciente evita ineficiências e garante melhores resultados computacionais. Assim, o presente relatório contribui para evidenciar os cenários ideais de aplicação de cada método, destacando o papel do *Quick Sort* e do *Heap Sort* como alternativas

robustas e eficazes para ordenação em larga escala.

7 REFERÊNCIAS BIBLIOGRÁFICAS

Referências

- [1] CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.
- [2] ZIVIANI, Nivio. **Projeto de Algoritmos: com implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2011.
- [3] DEVMEDIA. **Algoritmos de ordenação: análise e comparação**. <https://www.devmedia.com.br/algoritmos-de-ordenacao-analise-e-comparacao/28261>.
- [4] SHELL, D. L. **A High-Speed Sorting Procedure**. Communications of the ACM, v. 2, n. 7, 1959.
- [5] MANZANO, José Augusto N. G. **Algoritmos: Lógica para Desenvolvimento de Programas**. 29. ed. São Paulo: Érica, 2013.
- [6] ASCENCIO, Ana Fernanda; CAMPOS, Edilene Aparecida V. **Fundamentos da Programação de Computadores: Algoritmos, Pascal, C/C++ e Java**. 3. ed. São Paulo: Pearson Prentice Hall, 2011.
- [7] SEDGEWICK, Robert; FLAJOLET, Philippe. **An Introduction to the Analysis of Algorithms**. 1. ed. Boston: Addison-Wesley, 1996.